UPPSALA
UNIVERSITET

Självständigt arbete i informationsteknologi
25 maj 2026

Version Compatibility in Distributed Systems

Enabling Cross-Version Communication in Broker-Mediated Publish/Subscribe Architectures

Viktor Kangasniemi

Civilingenjörsprogrammet i informationsteknologi

Master Programme in Computer and Information Engineering

UPPSALA
UNIVERSITET

Institutionen för informationsteknologi

Besöksadress:
Ångströmlaboratoriet
Lägerhyddsvägen 1, hus 10

Postadress:
Box 337
751 05 Uppsala

Hemsida:
https://www.it.uu.se

Abstract

Version Compatibility in Distributed Systems

Enabling Cross-Version Communication in Broker-Mediated Publish/Subscribe Architectures

Viktor Kangasniemi

Broker-based publish/subscribe systems used in safety-critical distributed systems often enforce interface compatibility by requiring communicating components to use identical interface definitions. This strict equality check makes every interface change require coordinated updates across all affected components, which is costly and often impractical.

This thesis investigates whether interface compatibility can instead be determined analytically at registration time and allow components with different interface versions to communicate safely. To address this, a C++17 prototype is designed and implemented that replaces the equality check with a pipeline that analyzes field-level differences between two interface definitions and compiles an adaptation plan that transforms incoming messages to match the subscriber's expected layout, reused for every subsequent message.

The prototype is evaluated through correctness and performance benchmarks. All correctness tests pass across the evaluated evolution scenarios. Registration completes in the microsecond range. Cost models derived for per-message adaptation show that different types of field evolution carry different runtime costs, and that the overhead is small in absolute terms at the nanosecond scale.

Extern handledare: Johan Fredriksson, Saab AB
Handledare: Gerogios Fakas, Uppsala University
Examinator: Anders Arweström Jansson

Sammanfattning

Mäklarbaserade publish/subscribe-system som används i säkerhetskritiska distribuerade system kräver ofta att kommunicerande komponenter har identiska gränssnittsdefinitioner. Denna strikta likhetskontroll innebär att varje gränssnittsändring måste koordineras över alla berörda komponenter samtidigt, vilket är kostsamt och ofta opraktiskt.

Denna avhandling undersöker om gränssnittskompatibilitet i stället kan fastställas analytiskt vid registreringstillfället och möjliggöra kommunikation mellan komponenter med olika gränssnittsversioner. För att undersöka detta designas och implementeras en C++17-prototyp som ersätter likhetskontrollen med en pipeline som analyserar skillnader på fältnivå mellan två gränssnittsdefinitioner och kompilerar en anpassningsplan som transformerar inkommande meddelanden till prenumerantens förväntade layout, återanvänd för varje efterföljande meddelande.

Prototypen utvärderas genom korrekthets- och prestandatester. Alla korrekthetskontroller godkänns för de utvärderade evolutionsscenarierna. Registreringstiden slutförs på mikrosekundnivå. Kostnadsmodeller som tagits fram för anpassning per meddelande visar att olika typer av fältevolution medför olika körtidskostnader, och att tillägget är litet i absoluta tal på nanosekundnivå.

Contents

1 Introduction

Distributed systems are often built from software components that are developed, deployed and updated independently. When one component is changed, its communication interface may also change, while other components may still rely on an older version of that same interface. This creates a version mismatch [1]: even if both components are correct in isolation, they may no longer agree on the structure, order or types of the data being exchanged.

This problem is especially important in heterogeneous and safety-critical systems, where components may run on different platforms and may not be upgraded at the same time. In domains such as aviation, defense and industrial automation, software changes are also constrained by assurance, validation and qualification processes [2]. A central challenge is therefore to determine when components using different interface versions can still communicate correctly, without requiring all affected components to be upgraded simultaneously.

This thesis is conducted in the context of an event-driven publish/subscribe system used in distributed defense systems. In this system, each communicating component uses an XML-based interface definition that specifies the messages exchanged in a domain, including field names, primitive types, byte order and constraints. Since this definition also determines the binary layout of each serialized message, a subscriber relies on its local interface definition when deserializing a payload received from a publisher. Before communication is allowed, clients register their interface definitions with a central broker. Event data is then exchanged directly between clients, with the broker's role ending at registration and routing coordination.

Figure 1 shows a simplified example using one message definition, `PosUpdate`, within its interface definition. A publisher uses version 2 of this message, which adds a `timestamp` field, while the subscriber still expects version 1 without it. Because the two versions define a different field layout, the subscriber reads the incoming bytes at the wrong offsets and may silently produce incorrect results without any indication that a mismatch has occurred.

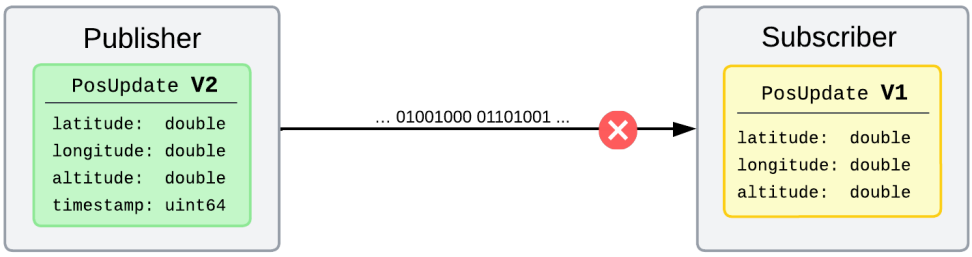

Figure 1 A publisher sends an updated version of `PosUpdate`, while the subscriber still expects an older version of the same message. As a result, the subscriber may incorrectly interpret the incoming data and silently malfunction.

It may well be that the subscriber does not need the `timestamp` field to fully operate. If that is the case, the fields that it actually requires, `latitude`, `longitude` and `altitude`, are still present in the publisher's layout, and the two components could in principle communicate correctly. The question is therefore when different interface versions can still communicate safely, and how such compatibility can be determined before data exchange begins.

The target system currently addresses this by enforcing strict interface equality at registration time. The broker accepts a registration only if the normalized interface definition is identical to the version already registered for that domain. This policy is simple and deterministic, and it prevents components with different binary layout assumptions from communicating. However, it is also restrictive: small interface changes prevent communication even when the involved message versions are structurally compatible.

This thesis investigates whether strict interface equality can be replaced by compatibility-aware interface analysis. The goal is to allow components using different versions of an interface to communicate when their differences are structurally compatible, while rejecting changes whose effects cannot be determined safely. Compatibility decisions must be explicit, deterministic and auditable before data exchange begins.

2 Background

This section introduces the technical concepts that form the foundation of this thesis. It begins with an overview of versioning and interface evolution in distributed systems, followed by a discussion of interface definitions, schemas and XML-based specifications. Schema and interface evolution is then presented, describing the ways in which definitions change over time and what those changes mean for communicating compo-

nents. The publish/subscribe communication model is introduced, with emphasis on broker-based architectures and domain-based routing. The section concludes with the compatibility terminology used throughout the thesis and the safety-critical constraints that motivate the design decisions in this work.

2.1 Versioning and Evolution in Distributed Systems

Distributed systems are built from independently developed and deployed components that communicate through defined interfaces [3]. Each component typically has its own development team, release cycle and deployment timeline. This independence is a core design principle that allows individual parts of a system to be maintained, upgraded and redeployed without requiring simultaneous intervention across the whole system.

Over time, components evolve. Requirements change, new capabilities are added, existing functionality becomes redundant and interfaces are revised. These changes are not exceptional. They are the ordinary consequence of software existing in a changing environment. The problem is that not all components are updated at the same time. Different versions of the same component may therefore be running simultaneously within a system, each reflecting different assumptions about the interfaces it shares with others.

This creates a version mismatch problem[1]. Two components that need to communicate may use different versions of the interface that defines their exchange. A system must therefore define how such mismatches are handled. Some systems require exact version equality, while others allow selected differences if they are considered compatible.

2.2 Interface Definitions and Schemas

This subsection describes how interfaces are defined in machine-readable form and how XML is used as a notation for such definitions. The term schema is closely related and is introduced here to clarify how it relates to interface definitions, since both terms appear in the literature and in the related work discussed in Section 4.

2.2.1 Interface Definitions

An interface definition is a machine-readable specification of the messages that a component can send or receive. It is expressed independently of any particular programming language, runtime environment or hardware platform. This makes interface def-

initions useful in heterogeneous systems, where communicating components may be implemented in different languages, run on different operating systems or execute on different hardware architectures. The interface definition names the messages in an interface, specifies the fields contained in each message and assigns a type to each field. Components can exchange data correctly when they use compatible interface definitions and therefore agree on the structure of the data being exchanged.

Many interface definition formats organize their content hierarchically. An *interface* groups together messages that belong to one logical communication endpoint. A *message* defines one structured unit of data that can be sent or received. A *field* defines one named and typed value within a message. This three-level structure of interface, message and field is common in systems where components communicate through explicitly defined data formats. Such machine-readable specifications are commonly referred to as Interface Definition Languages (IDLs), a term used across many distributed systems standards [4].

The term *schema* is closely related to interface definition. An interface definition usually refers to the complete machine-readable artifact that defines the messages exchanged between components. A schema refers more specifically to the structural and typing rules that determine how message data is represented, validated and serialized. In many systems, these concerns are expressed in the same source file. Therefore, the terms interface definition and schema are sometimes used interchangeably.

Listing 1 illustrates a common hierarchy for an interface definition.

```
Interface "SensorData"   (name, version)
  Message "Reading"      (id, direction)
    Field "SensorId"     : UInt32
    Field "Value"        : Float32
    Field "Timestamp"    : UInt32
  Message "Status"       (id, direction)
    Field "NodeId"       : UInt16
    Field "Healthy"      : UInt8
```

Listing 1: The three-level hierarchy of an interface definition.

2.2.2 XML-Based Interface Definitions

XML is a general-purpose markup language for representing structured data [5]. It is not an interface definition or a schema by itself. Instead, XML is a notation that can be used to express such specifications. An XML-based interface definition describes

interfaces, messages, fields and related metadata in a structured textual format.

XML is well suited to this purpose because it naturally represents hierarchical data. An interface can be represented as a top-level element, messages as nested elements and fields as elements within each message. Additional metadata, such as version information, message identifiers, field types and byte order, can be represented as attributes. This makes the interface definition readable for developers and processable by tools.

```
<Interface name="SensorData" version="1.0" byte_order="
   BIG_ENDIAN">
  <Message name="Reading" id="1" direction="out">
    <Field name="SensorId"  type="UInt32"/>
    <Field name="Value"     type="Float32"/>
    <Field name="Timestamp" type="UInt32"/>
  </Message>

  <Message name="Status" id="2" direction="out">
    <Field name="NodeId"  type="UInt16"/>
    <Field name="Healthy" type="UInt8"/>
  </Message>
</Interface>
```

Listing 2: Example of an XML-based interface definition.

Listing 2 shows a simplified XML-based interface definition. The top-level element defines the interface name, version and byte order. Each `Message` element defines one message in the interface, and each `Field` element defines one named and typed field within that message. A document of this form can serve both as developer-readable documentation and as input to tools that generate serialization code.

2.2.3 Code Generation and Serialization

In systems that use XML-based interface definitions, a code generator connects the specification to the runtime implementation. The XML file defines the structure of the messages, while the generated code implements the logic needed to serialize and deserialize those messages.

The pipeline consists of three main stages. First, the XML document is parsed into an internal representation of the interface. This representation contains the messages in the interface and the fields within each message, including their names, types, order and encoding-related metadata. Second, the code generator produces language-specific

code from this representation. In embedded systems, this code is often generated as C++ classes or functions that read and write message fields in a defined order. Third, the generated code is used at runtime. The sending component serializes a message into a binary payload, and the receiving component deserializes the payload using compatible generated code.

Figure 2 illustrates this pipeline. This pipeline makes the interface definition a central artifact in the communication system. If both communicating components use code generated from compatible interface definitions, they are expected to interpret the binary payload in the same way. However, if the components use incompatible versions of the interface definition, the same sequence of bytes may be interpreted differently. For example, a field may be read with the wrong type, at the wrong position or with the wrong byte order. Such mismatches can lead to incorrect runtime behavior even when both components compile successfully.

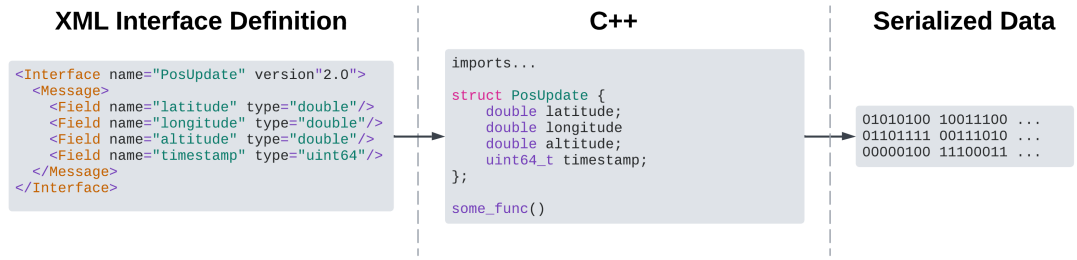

Figure 2 The XML-to-binary pipeline: an XML interface definition is parsed by a code generator into C++ serialization classes, which are then used at runtime to write and read binary payloads.

2.3 Schema and Interface Evolution

Schema changes can be categorized as structural or semantic. A structural change affects the binary layout of a message: fields may be added, removed, reordered, or assigned new types. These changes are visible in the schema definition and can be detected by comparing two versions. A semantic change affects the meaning of a field without changing its layout. For example, a field may change its unit or value interpretation while its name, type, and byte offset remain unchanged. A compatibility mechanism that compares schema definitions can therefore reliably detect structural differences. Semantic compatibility, however, generally cannot be determined from the schema alone and often requires human interpretation or domain knowledge.

Given a defined interface, the ways in which its schema can change between versions form a set of common structural evolution cases. Understanding these cases is useful when analyzing whether two versions can safely interoperate. Listing 3 shows two simple examples of structural evolution: adding a new field and changing the type of an existing field.

```
<!-- Version 1 -->
<Message name="Status">
  <Field name="id" type="UInt32"/>
</Message>

<!-- Field addition -->
<Message name="Status">
  <Field name="id" type="UInt32"/>
  <Field name="value" type="Float32"/>
</Message>

<!-- Type change -->
<Message name="Status">
  <Field name="id" type="UInt16"/>
</Message>
```

Listing 3: Examples of structural schema evolution cases.

This thesis mainly focuses on four structural evolution cases:

- **Field addition** occurs when a new field is added to a message in a later version. Components using an earlier version do not know that the field exists.
- **Field removal** occurs when a field that existed in an earlier version is absent in a later version. Components that still expect the field may be unable to recover its value.
- **Field reordering** occurs when the same fields appear in both versions but in a different sequence. In position-dependent encodings, this changes byte offsets and can lead to incorrect interpretation.
- **Type change** occurs when a field changes from one type to another. One common case is type widening, where the new type can represent more values than the old type, for example changing `UInt8` to `UInt16`. The opposite case is type narrowing, where the new type has a smaller representable range.

2.4 Publish/Subscribe Communication

Publish/subscribe (pub/sub) communication decouples message writers from message readers [6]. A writer, called a publisher in pub/sub systems, emits messages on a named channel without knowing which readers will receive them. A reader, called a subscriber, registers interest in a channel and receives messages that arrive on it without knowing which writer sent them. This decoupling simplifies system construction and allows participants to be added, removed or replaced without modifying the components they communicate with.

2.4.1 Broker-Based Architectures

In a broker-based publish/subscribe architecture, a central broker mediates communication between publishers and subscribers [6]. In the most common form, publishers send messages to the broker and the broker forwards each message to the subscribers that have registered interest in the corresponding channel. The broker therefore acts as a coordination point for matching writers and readers and as a relay for all message traffic.

Figure 3 illustrates this centralized communication pattern. Because the broker observes registrations before data exchange begins, it can also be a natural place to enforce admission rules before any data exchange begins.

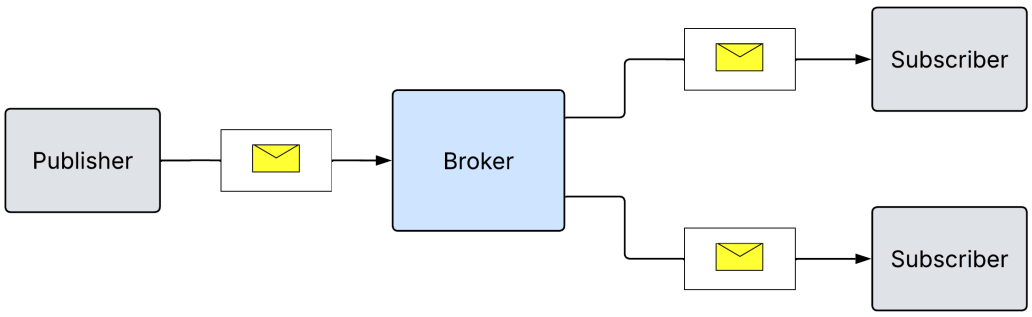

Figure 3 Broker-based publish/subscribe communication, where publishers send messages to a broker and subscribers receive messages through the broker.

In some broker-based systems, the broker's role is limited to the registration and coordination phase. The broker handles admission control, validates that participants meet the

required conditions and distributes routing information so that publishers know which subscribers to reach. After this phase, message data flows directly between publishers and subscribers without passing through the broker. This separates the control plane, where the broker enforces rules and manages participation, from the data plane, where events are delivered.

Figure 4 illustrates this coordination-only pattern. The broker remains a natural enforcement point for compatibility in this model, since it still observes both sides at registration time before any data exchange begins.

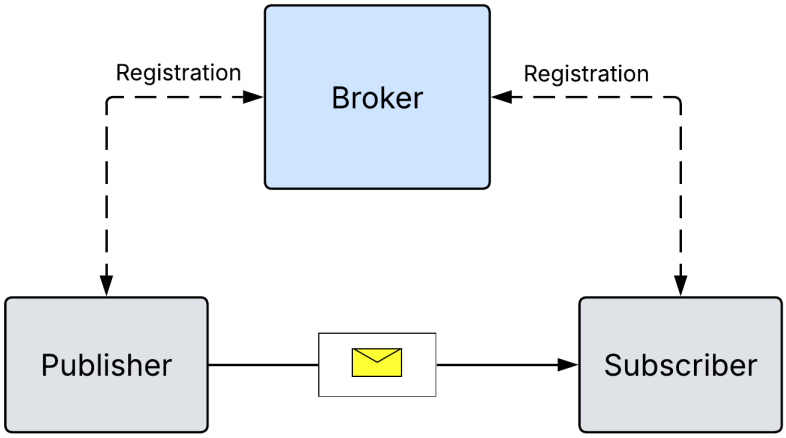

Figure 4 Coordination-only broker model, where the broker handles registration and distributes routing information but does not relay event data. After registration, events flow directly between publisher and subscriber.

2.4.2 Topic and Domain-Based Routing

Publish/subscribe systems commonly organize communication through named topics or domains. A topic is a logical channel for one type or group of messages. Publishers write messages to a topic, while subscribers register interest in the topics they want to receive.

A domain is a broader grouping of related communication. It may contain one or more message types and may also define shared rules for participants within that group. In typed systems, a topic or domain is often associated with an interface definition, such as the XML-based example shown in Listing 2, that describes the messages exchanged

through it. For example, a domain can group messages that belong to the same subsystem or functional area.

This organization allows publishers and subscribers to communicate through named groups instead of directly addressing each other.

2.5 Types of Compatibility

When two components use different versions of the same schema or interface definition, their relationship can be described using three broad compatibility terms[7]. These terms describe which side has changed and whether data produced using one version can still be interpreted correctly by the other.

- **Backward compatibility** means that a newer reader can correctly interpret data written using an older schema or interface version. This allows the reading side to be upgraded before the writing side.
- **Forward compatibility** means that an older reader can correctly interpret data written using a newer schema or interface version. This allows the writing side to be upgraded before the reading side.
- **Full compatibility** means that both directions are supported. A newer reader can read older data, and an older reader can read newer data. This is the strongest form of compatibility and is usually the hardest to guarantee.

In this thesis, compatibility is mainly considered in the writer-to-reader direction. This means asking whether data produced by a writer can be interpreted by a reader that may use a different interface version. In a publish/subscribe system, the writer corresponds to the publisher and the reader corresponds to the subscriber. Depending on which side uses the newer version, this direction may correspond to either backward or forward compatibility.

2.6 Safety-Critical Constraints on Interface Evolution

In safety-critical systems, interfaces between components are subject to strict specification and verification requirements [2]. One way to enforce these requirements in distributed systems is to require that communicating components use identical interface definitions. This is a design choice rather than a standards requirement. If two components use different interface definitions, the system cannot automatically know whether

one component will interpret the other's data correctly, and rejecting mismatches is a simple and conservative way to prevent unintended behavior.

Under safety-related software lifecycle standards such as IEC 61508, any modification to a qualified software component may require renewed verification activities to maintain its safety case. A component may be approved only for a specific version of the software and the interface definitions it was built with. Even a small interface change can create a new configuration that requires renewed review, testing or certification.

Common versioning solutions, surveyed in Section 4, take different approaches. Some allow unknown fields, some compare schemas when a message is read, and some check compatibility when components first connect. In a safety-critical context, any such approach must make its compatibility decisions visible and possible to review before deployment. A more flexible system must therefore define which differences are allowed, what data may be ignored or substituted, and when a connection must still be rejected.

It should reject unclear changes, behave consistently for the same pair of interface definitions, and make the decision rules explicit enough to inspect and justify. This ensures that flexibility is introduced through explicit decisions rather than hidden assumptions.

3 Purpose and Research Questions

This section defines the direction and scope of the thesis. It first states the purpose of the work and explains the problem that the thesis addresses. It then presents the research questions that guide the design and evaluation. Finally, it defines the delimitations of the study, including which parts of the target system, compatibility problem and evaluation are outside the scope of the thesis.

3.1 Purpose

The purpose of this thesis is to investigate how interface evolution can be handled more safely and flexibly in distributed systems that operate under strict safety and certification requirements. In many such systems, communication between components depends on interface definitions that must match exactly before data exchange is permitted [2]. While this approach ensures consistency, it also makes the system rigid and costly to evolve, since any interface change requires all affected components to be updated simultaneously.

This thesis therefore investigates whether interface compatibility can be determined in a

more meaningful way than by direct textual equality. Rather than requiring interfaces to be identical, the work explores whether a broker can analytically determine compatibility at the time two components register to communicate, based on whether the structural differences between their interface definitions would affect the correctness of the data exchange. The compatibility question is framed as writer-to-reader, asking whether a subscriber can correctly interpret the data produced by a publisher when the two sides hold different interface definitions.

To make this concrete, a compatibility analysis pipeline is designed and implemented as a prototype within an existing broker-mediated publish-subscribe system. The pipeline replaces the strict equality check performed at domain registration with a field-level analysis that classifies each difference between two interface versions and determines whether it can be handled safely. When a compatible pair is found, the pipeline compiles an adaptation plan that describes exactly how each field of an incoming message must be transformed to satisfy the receiving component's expected layout. This plan is then applied on every received message without repeating the analysis.

The overall goal is to demonstrate that a broker can make compatibility decisions analytically, compile the result into an executable plan and deliver that plan at a cost that is acceptable within the performance requirements of the target system. If successful, this approach would allow components with different interface versions to communicate without requiring coordinated upgrades, while keeping the compatibility rules explicit, reviewable and deterministic.

3.2 Research Questions

The main research question is:

> **MRQ:** How can a brokered publish-subscribe system support communication between components that use different versions of XML-based interface definitions?

The question is divided into three subquestions:

1. **RQ1:** How can compatible and incompatible interface changes be identified analytically at registration time?
2. **RQ2:** Which interface evolutions can be supported while still allowing subscribers to correctly interpret publisher payloads?

3. **RQ3:** What setup and per-message costs are introduced by supporting compatible interface evolution, and are these costs acceptable for the target system?

3.3 Delimitations

The evaluation measures the two main phases of the proposed mechanism separately. Registration-time overhead is measured through a network simulation layer that replicates the broker registration round-trip over UDP, including message framing and acknowledgement. This captures the delay a publisher observes before being permitted to exchange data. Per-message adaptation overhead is measured in-process to isolate payload transformation cost from transport effects.

The evaluation considers communication between one publisher and one subscriber. This setup is sufficient for evaluating the core compatibility analysis and adaptation logic. Performance effects related to fan-out, multiple subscribers with different resolution plans, and large-scale plan management are left for future work.

The compatibility analysis does not cover all possible forms of interface evolution. Instead, it focuses on a selected set of common structural changes. Semantic changes, byte-order changes, enumeration modifications and complex nested structures are outside the evaluated scope.

Hardware-specific constraints such as memory, processor speed and real-time scheduling are outside the evaluated scope; the measurements reflect mechanism overhead rather than platform-tuned performance.

4 Related Work

This section surveys existing approaches to interface versioning and schema compatibility in distributed systems. Each approach is examined for how it handles version differences between communicating components and where it falls short in the context of this work. The section closes with a summary of the identified gap that motivates the design presented in Section 7.

4.1 Protobuf

Protobuf, developed by Google, is a language-neutral platform for defining and exchanging structured data [8]. It uses `.proto` files to describe message types, fields and the numeric tags that identify each field. These definitions are used to generate serialization and deserialization code for different programming languages. Messages are then encoded in a compact binary format.

The numeric field tags are central to Protobuf's compatibility model. Each field is identified by its tag rather than by name or position, which allows many schema changes to be handled without requiring an explicit compatibility analysis step. For example, a newer writer may add a field that an older reader does not know about. When the older reader encounters the unknown tag, it can skip the field and continue decoding the rest of the message. Compatibility is therefore embedded in the encoding rules themselves, with no broker or separate analysis step involved, as long as incompatible changes such as reusing field numbers with different meanings are avoided [8].

The limitation in the context of this work is that Protobuf requires all participants to adopt its schema language, generated serialization code, and tag-based encoding. It cannot be applied directly to systems where the binary layout is defined by an external specification, such as an XML interface definition, and generated independently. Introducing tag-based encoding into an existing binary protocol would require modifying the serialization layer of every component in the system, which is outside the scope of this work.

4.2 Apache Avro

Apache Avro is a data serialization system that supports schema evolution through writer/reader schema resolution [9]. During deserialization, the reader uses both the schema used by the writer and the schema expected by the reader. A resolution process matches fields between the two schemas by name and determines how the writer's payload should be interpreted as the reader's expected representation.

Avro supports several forms of schema evolution. Fields may be reordered because matching is based on field names rather than position. Compatible type promotions, such as `int` to `long` or `float` to `double`, may also be applied. If a field is expected by the reader but absent from the writer schema, Avro uses the default value declared in the reader schema. If no suitable default is available, schema resolution fails. Avro also supports renaming through aliases, where the reader schema declares former names for fields or types and the resolution process treats them as equivalent [9].

This approach is conceptually close to the field-level adaptation used in this work. Both compare two schema versions and derive a per-field mapping between them. The key difference is where this resolution occurs. In Avro, schema resolution is performed by the reader during deserialization. In this work, compatibility is analyzed by the broker during registration, and the resulting adaptation plan is sent to the subscriber for use during message processing.

Avro therefore requires the writer schema to be available to readers at runtime, commonly through a schema registry or an equivalent mechanism. Unlike this work, Avro does not define broker-mediated interface policy enforcement during registration.

4.3 DDS-XTypes

DDS-XTypes is a standard from the Object Management Group that extends the Data Distribution Service (DDS) with formal type compatibility rules [10]. It defines extensibility kinds including `FINAL`, `APPENDABLE` and `MUTABLE`, which control how a type may evolve. `APPENDABLE` types allow new fields to be added without breaking existing readers, while `MUTABLE` types permit reordering because members are identified by a numeric member ID rather than by position. Within `MUTABLE` types, individual members may be marked optional, meaning that their absence from a received payload is acceptable and the reader treats the member as not present.

DDS is built around a global data space model [11]. Applications publish and subscribe to data through DDS middleware running in each participating process, rather than through a central broker placed between endpoints. The middleware handles discovery, type matching, serialization and data exchange. During writer and reader matching, DDS-XTypes allows the middleware to check whether the writer type and reader type are assignable. If the assignability rules are satisfied, communication is permitted even when the type definitions differ.

DDS-XTypes is the closest related approach to the problem addressed in this thesis. However, it is part of DDS itself rather than a separate compatibility mechanism. It therefore cannot be added directly to an existing broker-based system without also adopting the DDS type system and communication model.

4.4 Summary and Identified Gap

The approaches surveyed in this section address schema compatibility at different layers. Protobuf embeds compatibility support in a tag-based encoding and therefore re-

quires all participants to use the Protobuf schema language and serialization. Avro performs writer/reader schema resolution during deserialization and requires the writer schema to be available to the reader at runtime. DDS-XTypes provides the closest related solution by defining type assignability rules for publish/subscribe systems, but it is integrated into DDS and cannot be applied directly without adopting the DDS type system and communication model.

Among the surveyed approaches, none combine broker-side compatibility analysis with a precomputed per-subscriber adaptation plan that operates on existing binary layouts without modifying the serialization layer. This is the gap that the mechanism described in this thesis addresses.

5 The Target System

This section describes the Data Distribution (DD) system, the target system for this work. Unlike conventional broker-based publish/subscribe systems where the broker relays messages, the DD system uses the broker only for registration and routing coordination. Event data flows directly between clients over UDP. Despite the similar abbreviation, the DD system is a proprietary communication system and should not be confused with the OMG Data Distribution Service discussed in Section 4.3. Understanding its architecture, interface format and registration behavior is necessary to follow the design and evaluation presented in the sections that follow.

5.1 Architecture

The DD system is a broker-based publish/subscribe communication system for distributed heterogeneous embedded and defense systems, built around two components.

DDServer is a central broker that manages client registrations, maintains a registry of active domains and their participants, and coordinates routing information between clients. It is the single authority on which clients are registered to which domains and what events they publish or subscribe to.

DDclient is a library embedded in each process. It provides the publish/subscribe API through which a process registers domains, declares itself a publisher or subscriber for specific events, and exchanges data with other processes. Each process typically hosts one DDclient instance. The system communicates over UDP and is designed for low-latency operation, with a target average message latency below one millisecond.

The communication layer is platform-agnostic by design, making the system well suited for heterogeneous deployments where DDServer and DDclient instances run on different operating systems and hardware platforms within the same network. Serialization uses explicit byte order declarations in the interface definition, ensuring correct data interpretation across nodes with different hardware byte orders.

Figure 5 summarizes a simplified view of this architecture with one publisher and one subscriber. In the full system, the same pattern extends to a larger distributed set of DDclient instances.

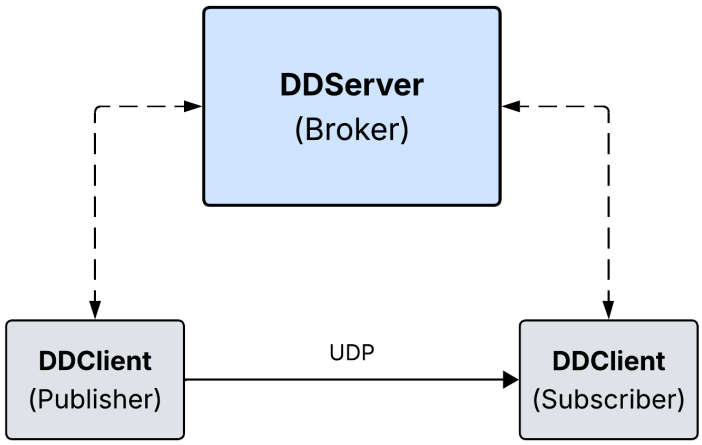

Figure 5 DD system architecture. DDServer acts as the central broker with DDclient instances embedded in each process. Solid arrows indicate the registration flow and dashed arrows indicate event delivery.

5.2 Domain Registration and Interface Validation

Clients are grouped into domains. A domain is a named collection of related messages described by a shared interface definition. Before a client can publish or subscribe to events in a domain, it must register that domain with DDServer.

During registration, the client sends either the full XML interface definition or an MD5 checksum of that definition to DDServer. The server stores the first XML definition it receives for a given domain name and uses it as the reference for all subsequent registrations of that domain. In practice this is the publisher, which typically registers a domain before any subscriber joins it. Before comparison, DDServer normalizes both

XML strings by removing content that does not affect the data model. This includes whitespace between elements, XML comments, processing instructions and description elements. The resulting canonical strings are then compared character by character.

If the strings match, the registration succeeds and the server responds with an acknowledgment that includes the numeric domain index assigned to that domain. If the strings do not match, the registration is rejected and the client is not permitted to join the domain. The registration exchange is illustrated in Figure 6.

5.3 Event Distribution

After registration, DDServer sends each publisher a broker information message specifying the network addresses of all current subscribers for each event. Publishers use this information to send events directly to each subscriber without involving DDServer in the delivery path. This peer-to-peer delivery model is the primary mode of operation. When subscriber lists change, DDServer sends updated broker information to the affected publishers so they can adjust their delivery targets accordingly.

A server-routed mode also exists, where a client sends an event to DDServer and the server forwards it to the relevant subscribers. This mode is used for test tooling and special-purpose scenarios and is not the primary delivery path in production deployments. The work described in this thesis concerns the peer-to-peer delivery path, where the broker's role ends at the registration and routing-information phases.

Figure 6 shows a simplified view of this communication flow, covering the registration exchange and peer-to-peer event delivery.

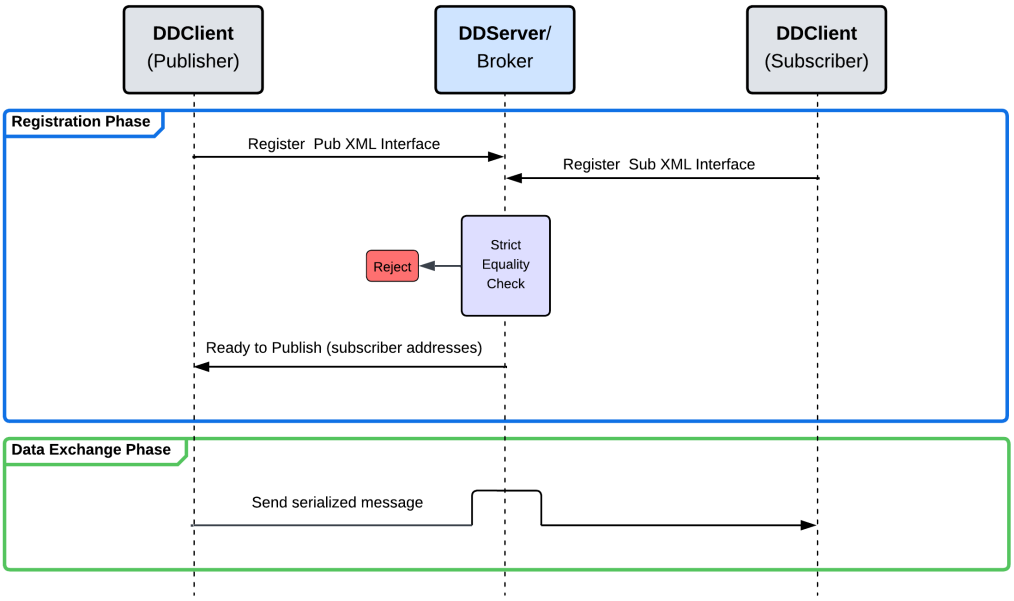

Figure 6 Simplified DD system communication flow. During registration, each client submits its XML interface definition to DDServer, which validates it and returns an acknowledgment or rejection. After registration, DDServer provides publishers with routing information so that event data is delivered peer-to-peer, without passing through the broker.

5.4 Interface Definition Format

The interface definition is the XML-based format used to specify a domain's interface. Each file describes one interface and declares the byte order, interface name, and interface version string. Within the interface, one or more `InterfaceMessage` elements define the individual messages. Each message is identified by a unique numeric event identifier and a name.

Each message contains an ordered sequence of field definitions. Table 1 summarizes the available field types by category.

The interface definition is consumed by a code generator that produces C++ serialization and deserialization code. The field order and types declared in the XML directly determine the binary layout of the serialized message, including each field's byte offset and width. The XML is therefore the authoritative binary layout contract for the interface. Any two components that share the same interface definition produce and consume binary payloads with identical field layouts. Appendix A.1 shows a complete

Category	Types
Integer primitives	Int8, Int16, Int32, Int64, UInt8, UInt16, UInt32, UInt64
String and array	FixedString, VariableString, UInt8Array
Composite	Repeating, Array, Define, Include
Constraint annotations	MinMaxRange, Default, EnumParser

Table 1 Field types grouped by category.

example covering all element categories listed in Table 1.

Listing 4 shows a trimmed interface definition illustrating the key structural elements.

```
<interface interfaceversion="3" interfacename="DD-BIT"
   byte_order="BIG_ENDIAN">
  <InterfaceMessage name="StatusReport" id="1001" direction="
   NOT_APPLICABLE">
    <UInt16 name="NodeId"/>
    <UInt32 name="StatusCode"/>
    <UInt8 name="Priority"/>
    <FixedString name="Label" length="16"/>
  </InterfaceMessage>
</interface>
```

Listing 4: Example interface definition showing the key structural elements.

5.5 The Strict Matching Policy

The registration step described in Section 5.2 implements a strict equality policy. A client may join a domain only if its interface definition is identical to the definition already stored by the server. This policy protects the system from runtime errors caused by binary layout mismatches. If two components use different layouts for the same event, one component may deserialize bytes according to the wrong field offsets or types. Such errors can lead to silent data corruption, since the payload itself does not contain enough information to detect the mismatch during message processing.

This safety comes at a high operational cost. Any change to an interface definition requires all publishers and subscribers in the affected domain to be updated to the same version before they can communicate again. This is difficult in the target deployment environment, where systems may be distributed across several nodes and upgraded at

different times. In defense systems, nodes may also be certified or qualified independently, so updating one software component can require additional validation of the platform on which it runs.

The strict policy therefore treats all interface changes as equally unsafe. A minor change, such as adding a field that no existing subscriber reads, causes the same rejection as a genuinely incompatible layout change. As a result, even structurally safe changes can force coordinated upgrades or temporary loss of communication. This limitation motivates the mechanism described in this thesis, which replaces strict equality with an analysis step that distinguishes safe interface evolution from changes that would corrupt or lose subscriber data.

6 Method

This section presents the research method used in the thesis. It explains the constructive approach used to design and implement the proposed mechanism, and describes how the evaluation is structured to measure correctness and performance within the assumptions of the target system.

6.1 Research Approach

This thesis follows a design science research approach [12], in which the main contribution is a designed artifact that addresses a problem in a defined practical context. Here, the artifact is a prototype that extends the existing interface registration model with compatibility analysis and payload adaptation. RQ1 and RQ2 are answered through the design and implementation, which defines how compatibility is determined and which evolutions are supported. The evaluation then validates these answers empirically. RQ3 is answered through performance measurement. Together, the three subquestions address the MRQ by demonstrating that the proposed mechanism can support communication between components using different interface versions in a brokered publish-subscribe setting.

This approach is suitable because the central question is practical. The thesis does not only ask whether interface compatibility can be analyzed in theory. It asks whether such analysis can be implemented in a brokered publish-subscribe setting and whether it can support communication between different interface versions at an acceptable cost.

The implementation is written in C++17. This choice reflects the target environment,

where the DD system and its clients are C++ codebases. It also allows the mechanism to be evaluated in a setting close to the intended implementation environment.

The work proceeds in three phases. The first phase analyzes the existing system to identify where strict interface matching is enforced and what a replacement mechanism must provide. The second phase designs and implements the compatibility analysis and adaptation mechanism. The third phase evaluates the mechanism against the requirements defined in Section 6.2.

6.2 Evaluation Requirements and Design

The evaluation is structured around four requirements derived directly from the research questions. Each requirement specifies what must hold for the mechanism to be considered correct and practical.

1. **Correct compatibility classification** (RQ1). The design defines analytically which interface changes are compatible and which are not. The evaluation validates this by checking that the mechanism produces the correct accept or reject decision for each tested evolution scenario. For each scenario, the expected result is defined manually before the test is run and is used to check whether the mechanism makes the correct decision.

2. **Correct payload interpretation** (RQ1, RQ2). Requirements 1 and 2 together answer RQ2 by showing which evolutions are accepted and that accepted cases produce correct results. When a compatible pair is accepted, the subscriber must recover the correct field values from the transformed payload. The design and implementation section describes how this transformation works. The evaluation validates it by placing known test values in the publisher payload and checking that the subscriber reads the expected values after the transformation is applied.

3. **Acceptable registration overhead** (RQ3). The proposed mechanism introduces additional work at registration time relative to strict matching, since it performs compatibility analysis and prepares transformation information for the subscriber. Because registration occurs once before data exchange begins, a higher one-time cost is tolerable. The evaluation measures this cost and compares it against the strict matching baseline. Acceptability is assessed against the target system's submillisecond latency requirement.

4. **Low per-message transformation cost** (RQ3). During data exchange, the subscriber applies transformation instructions to every received payload. This cost is

more sensitive than registration overhead because it accumulates with each message. The evaluation measures how this cost scales with the number and type of field differences between two interface versions, and derives a cost model for estimating the overhead of a given interface pair. Acceptability is assessed against the target system's latency budget.

These requirements are evaluated through three benchmark suites.

The correctness benchmark addresses requirements 1 and 2. It runs the compatibility pipeline over a set of interface pairs covering the evolution scenarios described in Section 2.3, checks each accept or reject decision against the expected outcome and, for accepted pairs, verifies that the adapted subscriber payload contains the correct field values.

The registration benchmark addresses requirement 3. Registration is measured through a network simulation layer that replicates the broker registration round-trip over UDP, including message framing and acknowledgement. This captures the delay a publisher observes before being permitted to begin data exchange, rather than measuring only the in-process analysis cost. Both the proposed mechanism and the strict matching baseline are measured under the same conditions.

The payload translation benchmark addresses requirement 4. It measures the per-message cost of applying the subscriber-side transformation to an incoming payload. The different types of field differences that can arise between two interface versions are benchmarked separately so that each contributes a known cost. A combined scenario covering multiple difference types is also measured. The publisher payload is serialized before measurement begins so that the measured cost reflects only the transformation step.

Both the registration and payload translation benchmarks use two categories of interface input. Isolated evolution scenarios contain a single type of interface difference per pair, making it possible to attribute measured cost to one cause. A mixed realistic interface pair combines several types of differences across a realistic number of messages, representing a more typical version update. Comparing these two input categories shows how cost scales with the number and variety of field operations in a plan.

7 Implementation

This section describes the design and implementation of DDCA (Data Distribution Compatibility Analyzer), the proposed mechanism that replaces the DD system's strict XML equality check with a compatibility analysis and payload adaptation pipeline. The

goal is to allow a publisher and a subscriber with compatible but non-identical interface definitions to communicate, without requiring coordinated upgrades.

7.1 Architecture Overview

DDCA is placed in the broker-side registration path. The publisher and subscriber provide their XML interface definitions during registration, and the broker uses DDCA to determine whether the two versions can communicate. If they are compatible, DDCA produces a resolution plan that is sent to the subscriber and later used during data exchange. Figure 7 shows where DDCA is placed in the communication architecture.

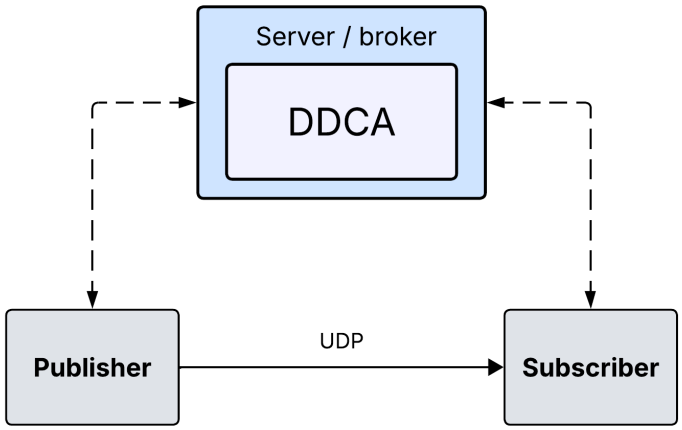

Figure 7 Placement of DDCA in the broker-mediated publish/subscribe architecture. DDCA runs during registration and produces resolution plans for compatible publisher-subscriber interface pairs.

After receiving the publisher and subscriber interface definitions, DDCA runs a pipeline of components inside the broker. The definitions are parsed into interface objects, analyzed for compatibility and, if accepted, compiled into a resolution plan. Figure 8 shows the internal DDCA pipeline.

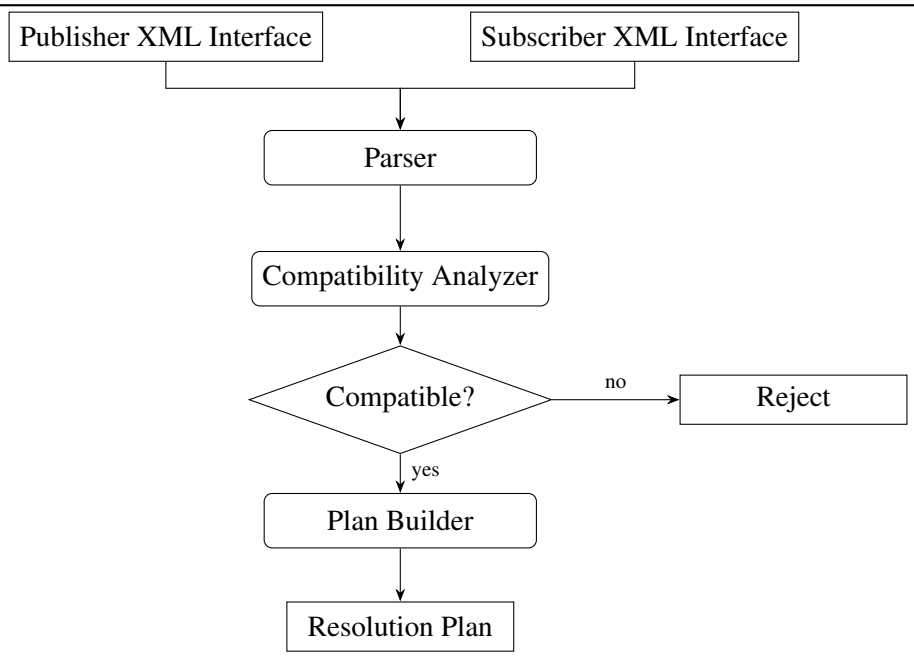

Figure 8 The DDCA pipeline inside the broker.

The DDCA communication flow is divided into the registration and data exchange phase. During the registration phase, the broker receives the publisher and subscriber interface definitions and runs compatibility analysis. If the pair is compatible, the broker builds one resolution plan per compatible message pair and delivers the plans to the subscriber. The subscriber stores the plans and replies with a plan acknowledgment. Only after this acknowledgment has been received does the broker mark the route as ready and notify the publisher. This ordering prevents the publisher from sending messages before the subscriber has the information needed to adapt and interpret them correctly. If compatibility analysis fails, the registration is rejected and the publisher and subscriber are not connected for data exchange.

During the data exchange phase, the publisher sends messages directly to the subscriber. On each received message, the subscriber looks up the stored plan for that message and applies it to translate the incoming payload into the expected subscriber layout. No further compatibility analysis is performed during data exchange.

Figure 9 shows both phases for a compatible interface pair.

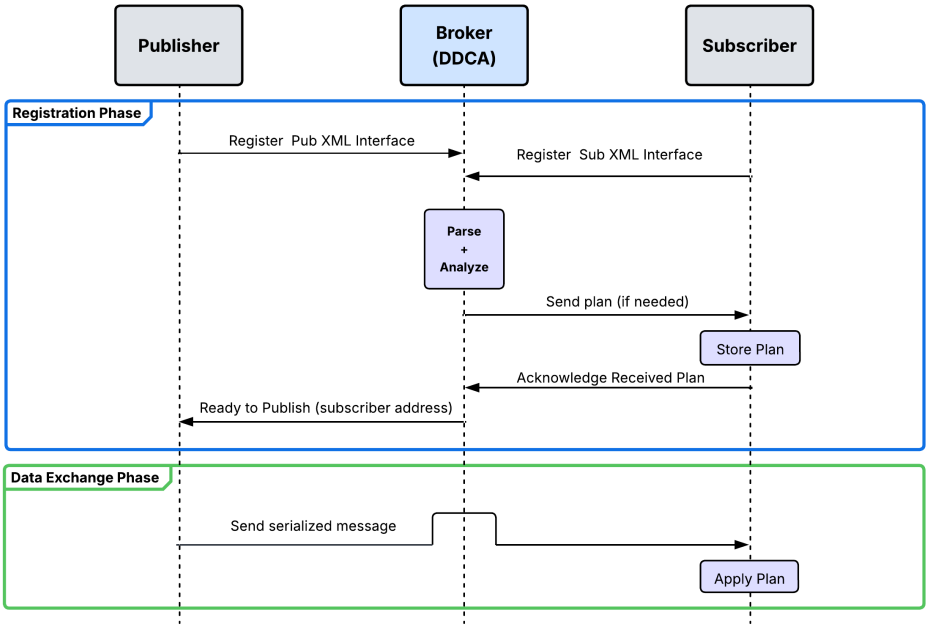

Figure 9 DDCA communication flow for a compatible interface pair. One resolution plan is delivered per message pair that requires adaptation.

7.2 Interface Parser

The first step in the DDCA pipeline is to parse the publisher and subscriber XML interface definitions. The parser converts these XML files into interface objects, which are the internal representation used by the compatibility analyzer and the resolution plan generator.

In the prototype, XML parsing is handled using the pugixml parser [13]. The parser reads the XML document tree in document order and extracts the interface structure, messages, and fields. It derives each field's type from its XML element name and reads additional metadata, such as field name, role, default value, and string length, from XML attributes. Unknown element names are skipped and reported with a diagnostic warning instead of causing parsing to fail. This allows the prototype to process interfaces that contain elements not supported by the current implementation.

7.2.1 Interface Object

The interface object is the structured form of an XML interface used by the later DDCA stages. In the prototype, it is implemented as a set of C++ structs. It stores the interface name, version, byte order, and ordered list of messages. Each message contains an ordered list of field descriptors.

A field descriptor stores the information needed to locate and interpret a field in the serialized payload. This includes the field name, primitive type, field size, byte offset, role, and optional default value. The byte offset is computed when the interface object is created by adding together the sizes of the fields that appear before it in declaration order. This matches the fixed-layout binary format used by the messages, where each field has a fixed position in the payload.

The computed byte offsets are stored directly in each field descriptor and are used by both the plan generator and the adapter to locate fields in a binary payload without re-parsing the XML. The full structure of the interface object is listed in Appendix A.2.

7.3 Compatibility Analyzer

After the parser has converted both XML definitions into interface objects, these objects are passed to the compatibility analyzer. The analyzer determines whether the two interfaces can communicate and produces a set of per-field resolution decisions that the plan builder uses to construct the resolution plan. Figure 10 gives a general overview of the analysis flow.

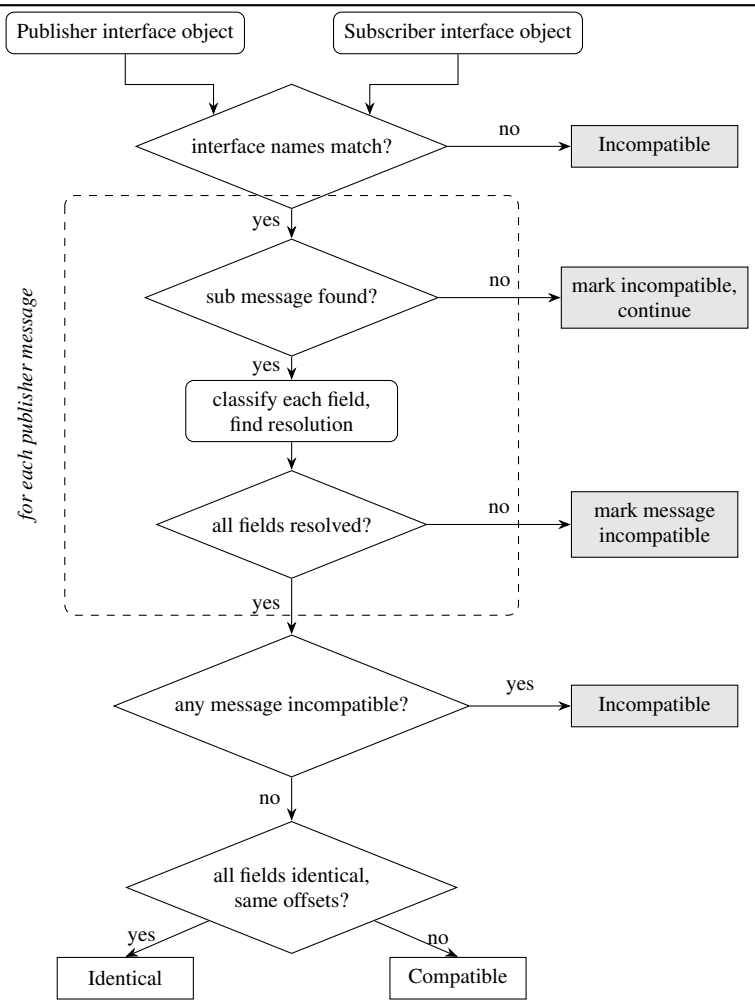

Figure 10 Flexible policy analysis flow. The dashed box repeats for each publisher message. Identical means the layouts match exactly and no plan is needed, Compatible means a resolution plan is built. Shaded nodes indicate rejection.

7.3.1 Message and Field Matching

Before fields can be compared, the analyzer must determine which publisher and subscriber messages correspond to each other. Messages are primarily matched by numeric message identifier. If no identifier match is found, the Flexible policy may use message-name matching as a fallback.

Fields within a matched message pair are matched by name rather than by position. This allows two interface definitions to list the same fields in different orders without causing a compatibility failure.

A consequence of name-based field matching is that field renaming is a breaking change. A renamed field has no match on the other side and is therefore treated as a removed field plus an added field. The normal missing-field rules then determine whether the pair is accepted or rejected.

The compatibility result applies to the interface pair as a whole. If any message in the interface cannot be adapted, the entire registration is rejected. An alternative design would accept the registration but route only the messages whose plans compiled successfully, silently dropping the rest. In a safety-critical context this introduces a different category of risk, because the subscriber would receive some events in the domain but miss others with no indication at the application level that certain message types are being withheld. Rejecting the registration outright makes the incompatibility visible and forces an explicit resolution before any data exchange begins.

7.3.2 Field Roles

To evaluate interface compatibility safely in a safety-critical context, DDCA must distinguish between fields that are required for correct interpretation and fields that can be handled more flexibly. For this reason, field roles are introduced as part of the interface model.

Fields in a message interface are not equally important for correct communication. Some fields must be present, while others may be optional or used only for diagnostic information. To represent this distinction, each field in an interface object carries a role annotation. The analyzer uses this role when a field is missing from one side of the connection.

Three roles are defined:

- **Required**: the field must be present in the publisher payload. If a required subscriber field is missing, the message pair is rejected.
- **Optional**: the field may be absent from the publisher payload if the subscriber declares a default value that can be injected during payload adaptation.
- **Diagnostic**: the field contains non-essential diagnostic information. If it is absent, the pair can still be accepted and no default value is injected.

Roles and default values are expressed as attributes on the field element in the interface and are a DDCA extension to the original format. A field with no `role` attribute is treated as required. An example of the extended format is shown in Appendix A.1.

7.3.3 Type Compatibility

When two fields are matched by name, the analyzer checks whether their types are compatible. Type compatibility is classified as identical, safely widenable, or incompatible. A safe widening means that the subscriber type can represent all values produced by the publisher

It accepts identical types and safe primitive widening. Narrowing conversions are rejected because the publisher value may not fit in the subscriber type. Conversions between signed and unsigned integer types are also rejected, even when the destination type is larger, because they may change the valid value range.

Each combination of matching and type compatibility results maps to one of the four field operations described in Section 7.4.1.

7.4 Resolution Plan Builder

The resolution plan is the compiled output of the registration phase. A *field operation* is the single entry in this plan: a self-contained instruction that gives the subscriber information what to do with one field at adaptation time. When the analyzer determines that a message pair is compatible, the plan builder translates the per-field resolution decisions into a flat, ordered array of field operations.

The operations follow the subscriber field order, so the plan can be processed sequentially to produce a payload in the layout defined by the subscriber interface. Each operation is populated once at plan-build time with the offsets, sizes, types or default bytes needed by the translator. As a result, the translator does not need to consult the original interface objects during message delivery.

One plan is built per compatible message pair. Each plan carries the publisher and subscriber message identifiers so the subscriber can look up the correct plan when a message arrives. Listing 5 shows the two structures that represent a field operation and a resolution plan.

```
ResolutionPlan:
    publisher_message_id
    subscriber_message_id
    operations: [
        FieldOp { opcode, src_offset, dst_offset, src_size,
   dst_size,
                  src_type, dst_type, default_bytes },
```

```
        ...
    ]
```

Listing 5: Resolution plan and field operation structures (pseudocode).

7.4.1 Field Operation Types

This thesis defines four field operation types to cover the interface evolution cases supported by DDCA. A field operation is one entry in a resolution plan. It stores an opcode and the metadata needed to correctly read the payload, such as offsets, field sizes, type identifiers, or precomputed default bytes. The full struct definitions are listed in Appendix B.

- **CopyBytes** is assigned when a field is present in both definitions with identical type and size. The operation stores the source offset, destination offset, and field size.
- **SkipSource** is assigned when the publisher contains a field that the subscriber does not define. The operation stores only the opcode, as no destination position exists in the subscriber layout.
- **InjectDefault** is assigned when the subscriber expects an optional field that the publisher does not send. Since default values in the IDD XML are stored as strings, the plan builder converts the declared default into a byte sequence once at plan-build time and stores it directly in the field operation.
- **ConvertPrimitive** is assigned when the subscriber expects a wider primitive type than the publisher sends. The operation stores the source offset, destination offset, source size, destination size, and the source and destination types.

7.5 Payload Adaptation

After the subscriber receives the resolution plans during the registration phase, they are stored in a plan cache indexed by the publisher and subscriber message identifier pair. When a publisher payload arrives, the subscriber uses this pair to look up the corresponding plan. The translator then allocates an output buffer sized to the subscriber payload and initializes it to zero. This gives a defined value to fields that are not written by any operation, such as missing diagnostic fields.

The translator then walks through the resolution plan sequentially. For each field operation, it reads the stored opcode and metadata and applies the corresponding adaptation step. `CopyBytes` and `InjectDefault` both copy a byte sequence into the output buffer. `SkipSource` performs no write because the publisher field has no position in the subscriber layout. `ConvertPrimitive` reads a primitive value from the publisher payload, widens it, and writes the converted value to the subscriber layout.

Primitive conversion is the only operation that requires interpretation of the stored bytes. The translator reads the source value in big-endian order, applies the widening conversion, and writes the result back in big-endian order. Signed integers are sign-extended, while unsigned integers are zero-extended. `Float32`-to-`Float64` widening reads the source bytes as a `float`, casts the value to `double`, and writes the result in the destination layout.

Since all compatibility decisions were made during registration, the translator does not parse XML, compare field names, or evaluate compatibility rules during message delivery. It only executes the precomputed resolution plan.

7.6 Compatibility Policies

Compatibility policies extend the core DDCA mechanism by making compatibility analysis configurable. The main pipeline consists of the parser, compatibility analyzer, resolution plan generator and payload adapter. A policy defines which field operations the analyzer may use and, therefore, which interface differences may be accepted. This allows different domains to apply different rules depending on their timing and safety requirements. This thesis implements two concrete policies.

- **Exact** reproduces the behavior of the existing DD system described in Section 5.2. The analysis pipeline is bypassed entirely: the two interface definitions are normalized and compared as strings. No field operations are generated and no adaptation takes place at runtime. The Exact policy serves as the performance baseline in the evaluation.
- **Flexible** runs the full analysis pipeline and permits all four field operations. It accepts any interface evolution case that can be expressed as a combination of those operations, including structural changes and safe primitive widening.

Table 2 summarizes which field operations each policy permits.

Field operation	Exact	Flexible
CopyBytes	Reject	Accept
SkipSource	Reject	Accept
InjectDefault	Reject	Accept
ConvertPrimitive	Reject	Accept

Table 2 Field operations permitted by each compatibility policy.

8 Evaluation

This section evaluates DDCA, the compatibility mechanism proposed in this thesis. The evaluation verifies that DDCA produces correct compatibility decisions and adapted payloads, and measures its performance during registration and payload adaptation.

8.1 Evaluation Setup

The evaluation uses three benchmark suites: correctness, registration and payload adaptation. The correctness benchmark checks whether the expected compatibility decisions are made and whether adapted payloads contain the correct field values. The registration benchmark measures the speed of the system during the registration phase. The payload adaptation benchmark measures the overhead of applying a resolution plan to translate incoming payloads during data exchange.

The evaluation uses two types of interface inputs. The first type is a set of isolated evolution scenarios, where all differences between the publisher and subscriber interfaces are of the same evolution type. For example, a scenario may contain several added fields or several fields with safe primitive widening, but it does not mix different evolution types. These scenarios are used in the correctness evaluation and in the payload adaptation benchmark. In the payload adaptation benchmark, they make it possible to measure the runtime cost of each field operation type separately.

The second type is a mixed-interface scenario, designed to resemble a more realistic interface update. It uses one larger XML interface pair where several compatible evolution types appear at the same time. This scenario is included to provide a more representative test result than the isolated scenarios.

All measurements were collected on a single Linux development machine using the C++ steady clock with nanosecond resolution. Each benchmark case ran 5 warmup iterations followed by 200 measured iterations. The warmup phase reduces cold-start effects, including instruction-cache initialization. The median of the 200 measured samples

is reported as the primary metric because it is robust to occasional outliers caused by scheduling jitter.

8.2 Correctness Evaluation

The correctness evaluation verifies two properties. DDCA must make the expected compatibility decision for each tested interface pair under the selected policy. For accepted pairs, the subscriber must read the correct field values from the adapted payload.

8.2.1 Compatibility Decisions

The benchmark evaluates 15 evolution scenarios under both the Exact and Flexible policies, giving 30 compatibility checks in total. The Exact policy accepts only identical interface definitions. The Flexible policy accepts all structurally compatible pairs where a sound adaptation plan can be produced, including cases where publisher extra fields are skipped, optional subscriber fields with declared defaults are injected and safe primitive widenings are converted. It rejects cases where no sound adaptation exists, including a required subscriber field the publisher does not send, unsafe type narrowing and cross-family type changes. Combination scenarios mixing multiple evolution types in one interface pair are also included. All 30 checks match the expected outcome. The complete per-scenario breakdown is listed in Appendix D.

8.2.2 Value Correctness

For each accepted interface pair, the subscriber must read the correct field values after adaptation. The benchmark covers all four field operation types in isolation and three combination scenarios, for example, `CopyBytes` combined with `InjectDefault` for an interface that both reorders fields and adds optional subscriber fields. The publisher serializes a payload with known test values, the subscriber applies the resolution plan and the benchmark checks each subscriber field for the expected value. All 39 checks pass. The complete check list is in Appendix D.

8.3 Registration Phase Performance

The registration benchmark evaluates the setup overhead introduced by DDCA before payload exchange begins. Registration is performed once when communication is es-

tablished and is not part of the per-message data path. The benchmark was executed in the network simulation using the registration flow described in Section 7.1. The measured interval starts when the publisher sends its registration request and ends when it receives the routing-ready confirmation. This captures the delay observed by the publisher before it is allowed to begin data exchange, including compatibility analysis, resolution-plan delivery when required and subscriber acknowledgement.

8.3.1 Registration Baseline Scaling

This benchmark measures registration time when the publisher and subscriber use identical interface definitions. The existing strict registration path is compared with the DDCA path under the Flexible compatibility policy. The strict path represents the approach currently used in the target system. In contrast, the DDCA path performs its own compatibility analysis. When DDCA determines that the interfaces are identical, no resolution plan is needed. The subscriber is therefore only notified that no plan is required.

The benchmark varies both the number of messages per interface and the number of fields per message. The tested interfaces range from small single-message definitions to larger multi-message definitions. For larger message counts, the maximum number of fields per message is reduced to keep the generated XML files within the size constraints of the network simulation.

This benchmark is performed to display the overhead of using DDCA compared with the existing strict registration approach. It also shows how the DDCA registration path scales with interface size. The purpose is to verify that registration remains practical and does not become unreasonably slow for the evaluated interface sizes.

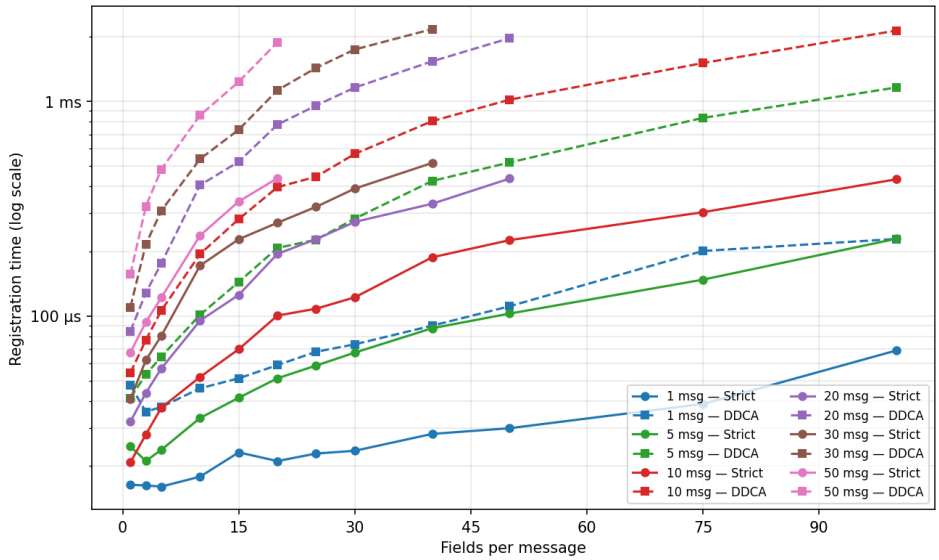

Figure 11 Registration time - DDCA vs Strict

Both paths increase roughly linearly as the number of fields in the interface increases, as shown in Figure 11. Across the evaluated range, DDCA takes three to five times longer than Strict. For the smallest interface, with one message and one field, registration takes about 16 µs with Strict and 48 µs with DDCA. For the largest evaluated interface, with 10 messages and 100 fields per message, the corresponding times are 433 µs and 2.1 ms.

8.3.2 Registration with a Mixed Evolved Interfaces

This benchmark measures registration time for a mixed evolved interface pair. The interfaces are designed to resemble a realistic version update, where one interface has evolved while remaining compatible with the other. The pair contains 20 messages and 231 declared fields on each side, and includes XML comments as found in real DD interfaces. It includes unchanged messages, publisher-only optional fields, subscriber-only optional fields with defaults, field reordering, safe primitive widening and one message that combines all supported evolution types. A summary of the mixed interface changes is listed in Appendix C.

The mixed evolved interface registers under the DDCA Flexible policy in approximately 967 µs. For comparison, an identical interface of comparable size registers in approximately 406 µs under the same DDCA path. The evolved case is slower because DDCA must analyze interface differences and prepare adaptation information. Since registration is performed once before data exchange begins, this remains a practical setup cost

for the evaluated case.

8.4 Data Exchange Phase: Payload Adaptation Performance

The payload adaptation benchmark measures the per-message cost of applying a compiled resolution plan to a publisher payload on the subscriber side. Registration, compatibility analysis and plan compilation are completed before the timed loop begins and are therefore excluded from the measurement.

8.4.1 Isolated Field Operations

Each benchmark case focuses on one field operation type at a time. For example, one case measures only `SkipSource` operations, while another measures only `ConvertPrimitive` operations. This makes it possible to see how much each operation type contributes to payload adaptation time. The number of adapted fields is then increased from 1 to 100 to show how the cost grows as the resolution plan becomes larger. A plain `memcpy` baseline is included to show the cost of copying a payload when no adaptation is needed.

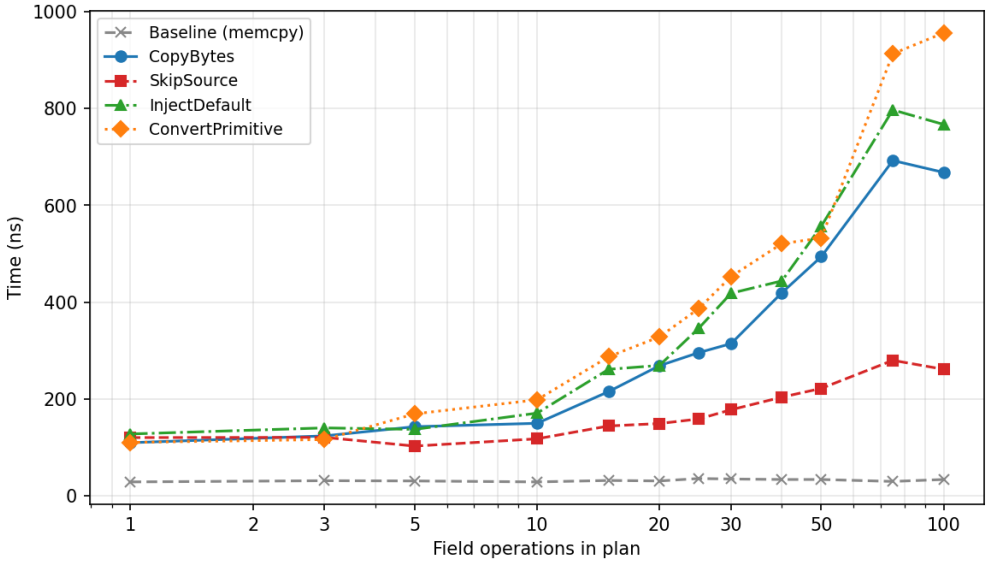

Figure 12 Median payload adaptation time per message by field operation type and evolved-field count. `BaselineMemcpy` shows raw `memcpy` cost for equivalent payload sizes.

Figure 12 shows that all field operations are slower than the `memcpy` baseline, which remains close to 30 ns across the evaluated range. At one adapted field, the field operations measure between 110 and 128 ns. As the number of adapted fields increases, the operation types separate more clearly. `SkipSource` scales the best, reaching approximately 262 ns at 100 fields. `CopyBytes` and `InjectDefault` reach approximately 668 ns and 767 ns, respectively. `ConvertPrimitive` is the most expensive operation, reaching approximately 956 ns at 100 fields.

8.4.2 Mixed Resolution Plan

This benchmark measures the per-message adaptation cost when the compiled resolution plan for a single message contains all four field operation types simultaneously. Message 1020 (MixedEvolution) in the realistic interface described in Section 8.3.2 contains five `SkipSource` operations, five `CopyBytes` operations, five `ConvertPrimitive` operations and five `InjectDefault` operations, giving a compiled plan with 20 field operations across all four types. Registration and plan compilation are performed once before measurement begins. Only the per-message adaptation step is measured.

The measured median per-message adaptation time is 177 ns for the full 20-operation plan. This serves as a reference for the cost of a mixed plan covering all supported evolution types simultaneously. The result shows that combining multiple operation types in one plan does not introduce additional structural overhead. The plan executor processes all operations in a single pass and pays the fixed setup cost once, so the cost of a mixed plan reflects one setup cost plus the per-field contributions of each operation, regardless of how many types are present.

8.5 Payload Adaptation Cost Model

From these results, a linear cost model can be derived for the data exchange phase. The model estimates the additional runtime cost per received message from the number and type of field operations in the resolution plan. The adaptation time is modeled as a fixed per-message cost plus a cost for each field operation in the resolution plan:

$$T_{\text{msg}}(n) \approx C_0 + c_{\text{op}} \cdot n \text{ ns}$$

where n is the number of field operations in the compiled resolution plan, C_0 is the fixed cost of invoking the plan executor for one message and c_{op} is the marginal cost added

by each field operation of the given type. Table 3 shows the fitted values for the linear cost model based on the benchmark data. The model estimates the total execution time as a fixed cost plus an additional cost for each field.

Operation	Fixed cost C_0 (ns/message)	Per-field cost c_{op} (ns/field)
SkipSource	115	1.8
CopyBytes	123	6.5
InjectDefault	137	7.5
ConvertPrimitive	132	9.1
BaselineMemcpy	31	0.0

Table 3 Payload adaptation cost model parameters fitted to benchmark data.

The fitted values show that the fixed cost C_0 dominates at small field counts and is comparable across all four operation types at approximately 115 to 137 ns. As more fields require adaptation, the per-field cost becomes the primary contributor. `ConvertPrimitive` has the highest per-field cost because it performs a numeric type conversion for each field. `SkipSource` has the lowest per-field cost because it advances the source offset without writing anything into the subscriber payload.

The model makes the runtime cost of interface evolution easier to reason about. A compatible interface change produces a resolution plan with a specific set of field operations. The cost model can therefore be used to estimate the added cost per received message from the number and type of operations in the plan.

8.6 Summary of Evaluation

All correctness checks pass across 15 evolution scenarios and two compatibility policies. DDCA makes the expected compatibility decisions and preserves the expected subscriber-side field values after payload adaptation.

The registration benchmark shows that DDCA costs approximately three to five times more than the strict path for identical interfaces across the evaluated range, with both paths scaling approximately linearly with total field count. For a realistic 20-message, 231-field mixed-evolution interface, DDCA registers in approximately 967 µs. This is roughly 2.4 times the DDCA registration time for an identical interface of comparable size. Since registration is a one-time setup cost, this overhead does not affect per-message throughput.

Payload adaptation adds a fixed cost of approximately 115 to 137 ns per received message and a per-field cost ranging from approximately 1.8 ns for `SkipSource` to approximately 9.1 ns for `ConvertPrimitive`. For a comprehensive mixed plan containing 20 field operations across all four types, the measured adaptation time is 177 ns, serving as a reference for the per-message cost when all supported evolution types are present in a single plan.

9 Discussion

This section interprets the evaluation results and positions them in relation to the thesis goals. It examines the trade-offs introduced by the proposed mechanism, considers what the results imply for deployment in safety-critical systems and identifies the limitations of the current prototype.

9.1 Correctness of the Compatibility Analysis

The evaluation confirms that DDCA produces the expected compatibility decision for every tested interface pair and that accepted pairs yield correct field values after adaptation. This holds across 15 evolution scenarios under both the Exact and Flexible compatibility policies. The result is encouraging, but it should be interpreted within its scope. The tested cases cover four isolated evolution types, field additions, field removals, reordering and safe primitive type widening, each also tested with specific variations such as fields carrying declared defaults, as well as combination scenarios. They do not represent the full space of possible interface changes, and more complex patterns may require additional compatibility rules or field operations to be handled correctly.

All tested scenarios assume that subscriber interfaces include DDCA role annotations. These annotations classify each field as required, optional or diagnostic. This allows the analysis to determine how important each field is for the subscriber and to make safe compatibility decisions. Without role annotations, DDCA cannot tell whether a missing field is essential for the subscriber or only used for diagnostics. In that case, the analysis must be conservative and reject interface pairs that differ. Fields may also include a default annotation, which provides a value when the publisher does not include the field. The evaluated scenarios all included these annotations. In a real deployment, however, introducing and maintaining them would be an important prerequisite for using the more flexible compatibility policies safely.

Beyond this prerequisite, the correctness results show that the mechanism holds at two

distinct levels. At the decision level, compatibility classification is accurate for the evaluated scenarios. At the execution level, the resolution plan produces the expected field values without corruption. Both levels must hold for the mechanism to be considered correct in practice, and the evaluation confirms that they do for the cases tested.

9.2 The Trade-off Between Strict Matching and Flexible Compatibility

Strict matching provides a strong guarantee through simplicity. When two components present identical interface definitions, no analysis is required and no risk of misclassification exists. Any version difference is treated as a potential layout conflict and rejected. This is not an unreasonable policy in a controlled environment where all components can be upgraded simultaneously and deployment is coordinated.

The need for flexible compatibility becomes clearer when these conditions do not hold. In distributed systems, components may be upgraded independently, maintained by separate teams or affected by different certification and qualification processes. In such cases, strict matching can create a coordination requirement that is difficult or costly to meet. For example, strict matching rejects a publisher that adds a diagnostic field, a subscriber that adds optional fields or two components whose field order differs for alignment reasons. These pairs are rejected even if the difference would not prevent the subscriber from interpreting the data correctly. The publisher and subscriber therefore cannot communicate until all affected components are updated together.

DDCA replaces the binary identical-or-different judgment with a field-level analysis that can distinguish safe structural differences from genuine incompatibilities. The cost of this is a more complex registration step and an added per-message overhead when adaptation is required. Whether that trade-off is favorable depends on the operational context. For systems where coordinated upgrades are the norm, strict matching remains the simpler and lower-cost choice. For systems where independent deployment is the reality, DDCA's one-time registration cost is likely a more acceptable price than broken communication.

9.3 Performance Implications for Deployment

Registration overhead and per-message adaptation cost affect the system in different ways and should therefore be evaluated separately. Registration is a one-time cost paid before any data exchange begins, and is paid once regardless of how many messages

are subsequently exchanged. Per-message adaptation cost is paid on every received message for the lifetime of the communication session. A higher registration cost is therefore more tolerable than an equivalent per-message cost.

The cost model derived from the benchmark data makes the per-message overhead concrete and actionable. A compatible interface pair produces a resolution plan with a specific set of field operations, and the model can be used to estimate the added per-message cost from the number and type of those operations before deployment. The runtime cost of accepting a version pair can therefore be estimated rather than treated as unknown. For example, accepting a type widening costs approximately five times more per field than accepting a dropped publisher field, and this difference can be predicted from the cost model before deployment. Two structurally compatible interface version pairs may therefore have different runtime costs, even though both are valid from a compatibility perspective. A binary compatible-or-incompatible decision would hide this difference.

It is also worth noting that the upper-bound test cases, where up to 100 fields have evolved simultaneously, represent stress tests rather than typical version increments. A change of that magnitude would likely be part of a broader system update rather than a routine interface revision. In contrast, a typical version update would usually affect only a small number of fields, keeping the per-message cost closer to the lower end of the evaluated range.

This highlights the role of DDCA's compatibility policies in managing the trade-off between performance and flexibility. A more conservative policy can limit which field operations are accepted and thereby reduce the per-message cost that an accepted version pair may introduce. For example, a deployment that cannot tolerate type-widening conversions can reject that evolution type while still allowing lower-cost changes, such as skipped publisher fields. If additional field operation types are introduced later, especially operations with higher runtime cost, the same benchmark infrastructure can be used to measure their cost before they are enabled by a policy.

The registration costs should be understood as a lower bound. The benchmark uses a simulated broker over loopback UDP rather than the real DD stack, and real deployment conditions, including concurrent registrations and network latency under load, would increase those numbers. The payload adaptation benchmark is measured in-process by design, since transport cost is present regardless of whether DDCA is applied. The cost model and benchmark results characterize the mechanism's own contribution to latency, not the full system overhead.

9.4 Implications for Safety-Critical Systems

Interface evolution is particularly sensitive in safety-critical systems because an incorrect interpretation of serialized data can affect system behavior in ways that are difficult to detect and attribute. This makes explicit, deterministic and auditable compatibility decisions more important than in general-purpose systems. DDCA does not remove the need for safety processes, certification activities or domain expertise, but it changes what those processes have to reason about.

Under strict matching, any version difference is rejected without further analysis or recorded reasoning. DDCA instead bases the decision on field-level analysis, which makes the result possible to inspect and justify. When an interface pair is rejected, the diagnostic output identifies the fields that caused the rejection and the type of change involved. This provides support for auditing and for justifying compatibility decisions as part of a safety case.

The role-aware field classification is a deliberate part of the DDCA design. Required fields are treated conservatively. If a required subscriber field is missing from the publisher interface, the pair is rejected because the subscriber depends on data that the publisher does not provide. Optional fields may be accepted when a safe adaptation is available. Diagnostic fields may be treated more flexibly under permissive compatibility policies, because they are not part of the core functional interpretation of the message. This distinction prevents DDCA from treating all interface differences in the same way. It is important because some fields may carry safety-relevant data, while others may only be used for diagnostics or monitoring.

The all-or-nothing acceptance rule follows the same principle. A registration is accepted only when every message in the interface can be mapped to a valid adaptation plan. Partial acceptance, where some messages are adapted while others are ignored, would create a risk that is harder to detect than a rejected registration. For example, a subscriber that assumes it receives all messages in a domain may behave incorrectly if some message types are silently omitted. An explicit rejection at registration time is safer because the problem is visible before data exchange begins. The all-or-nothing rule therefore supports deterministic behavior and makes the compatibility decision auditable at the interface level.

9.5 Limitations

The most significant limitation is that DDCA is a standalone prototype and has not been integrated into the real DD system. The benchmarks measure the compatibility

analysis and adaptation mechanism in isolation using a network simulation layer. The registration costs therefore represent a lower bound on real-world overhead. Behavior under actual system conditions, including concurrent registrations, real network latency and deployment constraints, remains to be evaluated.

The evaluation also considers only a single publisher-subscriber pair at a time. A distributed system typically involves multiple concurrent publishers, subscribers and interface versions. How registration overhead and adaptation cost scale with the number of concurrent nodes, and whether the broker or subscriber become bottlenecks under load, is not captured by the current benchmarks.

The adaptation mechanism also relies on a static-offset model. It assumes that each field has a fixed byte offset that can be computed from the interface definition alone. This holds for the primitive types evaluated in this thesis, but not for variable-size fields such as `VariableString` or for `Repeating` blocks. In these cases, later field offsets depend on runtime values, such as a string length or element count in the payload. Supporting such types would require the adapter to read runtime values and update offsets dynamically during adaptation. This is outside the current static plan model and would need to be addressed before DDCA could support the full interface format.

Finally, the compatibility model is intentionally limited to structural compatibility. Semantic changes, where a field retains the same name and type but changes its meaning or unit, cannot be detected from the interface definition alone and are therefore out of scope. Field renaming is similarly not handled. DDCA matches fields by name, so a renamed field is treated as a missing field on one side and an unknown field on the other. This could be resolved by field aliases, where a field declares its former names, allowing the matching logic to recognize the rename without structural changes. This extension is not implemented as part of this work.

10 Conclusions

This thesis investigated whether a broker-mediated publish/subscribe system can support communication between components that use different versions of XML-based interface definitions. The investigation produced DDCA, a prototype compatibility and adaptation mechanism that replaces strict interface equality at registration time with field-level compatibility analysis. DDCA classifies interface differences, determines whether a safe adaptation can be derived, and generates a resolution plan that the subscriber uses during message processing.

10.1 Summary of Contributions

The main contribution is the design and implementation of DDCA as a prototype for a broker-based publish/subscribe system. DDCA parses XML interface definitions into internal interface objects, analyzes compatibility using field names, types and declared field roles, and generates a resolution plan consisting of field operations. During the data exchange phase, the subscriber uses this compiled plan to adapt incoming publisher payloads into the expected subscriber layout. The prototype also provides configurable compatibility policies, making it possible to control which interface differences are accepted according to the requirements of the system.

10.2 Research Question Answers

The three research questions are answered below based on the design, implementation and evaluation presented in this thesis.

RQ1.

How can compatible and incompatible interface changes be identified analytically at registration time?

DDCA identifies compatible and incompatible interface changes at registration time by parsing both interface definitions into interface objects, matching corresponding messages and fields, and classifying each field difference as a supported or unsupported evolution. All compatibility checks across the evaluated evolution scenarios produce the expected result, confirming that analytical compatibility detection is feasible for the evaluated patterns.

RQ2.

Which interface evolutions can be supported while still allowing subscribers to correctly interpret publisher payloads?

Interface evolution cases are supported by mapping their structural differences to field operations under the selected compatibility policy. Publisher-only fields that can be ignored are handled by `SkipSource`, subscriber-only optional fields with declared defaults by `InjectDefault`, compatible fields with different offsets by `CopyBytes`, and safe primitive widenings by `ConvertPrimitive`. A single message may require

several operation types simultaneously. All value correctness checks pass, confirming that adapted payloads contain the expected field values for all four operation types and for the evaluated combination scenarios.

RQ3.

What setup and per-message costs are introduced by supporting compatible interface evolution, and are these costs acceptable for the target system?

Registration introduces a one-time setup cost during the registration phase. Compared to the strict path, DDCA registration takes approximately three to five times longer for identical interfaces. For the evaluated interface sizes, registration times remain in the sub-millisecond to low-millisecond range. This cost is paid before data exchange begins and therefore does not increase per-message latency during the data exchange phase.

Payload adaptation adds a small fixed cost per received message and an additional cost per adapted field, which varies depending on what type of interface evolution is handled. Registration is a one-time cost that does not affect per-message throughput, and per-message adaptation overhead is small in absolute terms at the nanosecond scale. Whether these costs are acceptable depends on the latency requirements of the target system and should be evaluated in that context.

10.3 Future Work

The most direct next step is integrating DDCA into the real DD system. This requires replacing the XML string equality check in the domain registration logic with the DDCA compatibility analysis pipeline, extending the registration protocol with a new message type to carry the resolution plan from the broker to the subscribing client before routing is confirmed, and integrating plan application into the client-side message handling where received payloads are currently deserialized. Validating the integrated mechanism under real network and deployment conditions would establish whether the measured costs hold at system scale. The adapter should also be extended to support variable-size types such as `VariableString` and `Repeating` blocks, nested structures, arrays and field aliases for rename handling. The evaluation could be expanded to include multiple publishers and subscribers using real project interface definitions. This would show how registration overhead changes when several nodes register concurrently. Compatibility policies could be extended to consider more than the type of interface change. For example, a policy could reject a version pair if the estimated adaptation cost is too high or if the affected topic is safety-critical. For safety-critical

deployment, future work should formalize the compatibility rules and integrate interface checking into development workflows such as continuous integration pipelines. A schema registry that stores known interface versions and caches resolved compatibility results could also reduce repeated registration overhead, avoiding re-running the full analysis when the same version pair is encountered again.

References

[1] H. Knoche and W. Hasselbring, “Continuous API evolution in heterogenous enterprise software systems,” in *2021 IEEE 18th International Conference on Software Architecture (ICSA)*, 2021, pp. 58–68.

[2] International Electrotechnical Commission, “IEC 61508: Functional safety of electrical/electronic/programmable electronic safety-related systems,” International standard, 2010, part 1 (IEC 61508-1:2010).

[3] A. S. Tanenbaum and M. Van Steen, *Distributed Systems: Principles and Paradigms*, 3rd ed. Pearson, 2017.

[4] Object Management Group, “Interface definition language (IDL) version 4.2,” OMG Specification, 2018, formal/18-01-05. [Online]. Available: https://www.omg.org/spec/IDL/4.2/PDF

[5] T. Bray, J. Paoli, C. M. Sperberg-McQueen, E. Maler, and F. Yergeau, “Extensible markup language (XML) 1.0 (fifth edition),” W3C Recommendation, 2008. [Online]. Available: https://www.w3.org/TR/xml/

[6] P. T. Eugster, P. A. Felber, R. Guerraoui, and A.-M. Kermarrec, “The many faces of publish/subscribe,” *ACM Computing Surveys*, vol. 35, no. 2, pp. 114–131, 2003.

[7] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA: O’Reilly Media, 2017.

[8] Google, “Protocol Buffers: Language guide (proto3),” Online documentation, 2025. [Online]. Available: https://protobuf.dev/programming-guides/proto3/

[9] Apache Software Foundation, “Apache Avro specification,” Online documentation, 2021. [Online]. Available: https://avro.apache.org/docs/1.11.1/specification/

[10] Object Management Group, “Extensible and dynamic topic types for DDS (DDS-XTypes), version 1.3,” OMG Formal Specification, Feb. 2020, document number formal/2020-02-04. [Online]. Available: https://www.omg.org/spec/DDS-XTypes/1.3/PDF

[11] Object Management Group, “Data distribution service (DDS) version 1.4,” OMG Specification, 2015, formal/2015-04-10. [Online]. Available: https://www.omg.org/spec/DDS/1.4/PDF

[12] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[13] A. Kapoulkine, *pugixml 1.15 Manual*, 2025, version 1.15. [Online]. Available: https://pugixml.org/docs/manual.html

A Interface Definition XML and Interface Object

A.1 Interface Definition XML Example

```
<?xml version="1.0" encoding="UTF-8"?>
<interface interfaceversion="2" interfacename="DD-EXAMPLE"
   byte_order="BIG_ENDIAN">

  <description>Template interface showing all element types
   .</description>

  <!-- Define: a named struct template inlined wherever
   Include references it -->
  <Define Name="TestEntry">
    <UInt16 name="TestId"/>
    <UInt16 name="Result"/>
    <UInt32 name="ErrorParameter"/>
  </Define>

  <!-- Primitive fields with constraint annotations -->
  <InterfaceMessage name="NodeStatus" id="1001" direction="
   NOT_APPLICABLE">
    <UInt8  name="NodeId"/>
    <UInt16 name="FaultCode"/>
    <Int16  name="Temperature">
      <MinMaxRange><min>-400</min><max>1500</max></
   MinMaxRange>
    </Int16>
    <UInt8  name="OperatingMode">
      <Default>0</Default>
      <EnumParser>
        <EnumValue value="0" name="IDLE"/>
        <EnumValue value="1" name="ACTIVE"/>
        <EnumValue value="2" name="FAULT"/>
      </EnumParser>
    </UInt8>
    <FixedString    name="NodeLabel"    length="16"/>
    <VariableString name="DiagMessage"  maxlength="64"/>
    <UInt8Array     name="RawPayload"   length="8"/>
  </InterfaceMessage>
```

```
  <!-- Repeating: variable-length array; element count given
   by a reference field -->
  <InterfaceMessage name="TestReport" id="1002" direction="
   NOT_APPLICABLE">
    <UInt16 name="NodeId"/>
    <UInt32 name="NrOfTests">
      <MinMaxRange><min>0</min><max>70</max></MinMaxRange>
    </UInt32>
    <Repeating name="TestResults" reference="NrOfTests">
      <UInt16 name="TestId"/>
      <UInt16 name="Result"/>
      <UInt32 name="ErrorCode"/>
    </Repeating>
  </InterfaceMessage>

  <!-- Include: inlines the Define block declared above -->
  <InterfaceMessage name="DiagRequest" id="1003" direction="
   NOT_APPLICABLE">
    <UInt16  name="RequestorId"/>
    <include name="TestEntry"/>
  </InterfaceMessage>

  <!-- Array: fixed-length array of a primitive type -->
  <InterfaceMessage name="CalibrationData" id="1004"
   direction="NOT_APPLICABLE">
    <UInt16 name="NodeId"/>
    <Array  name="Coefficients" type="UInt32" length="8"/>
  </InterfaceMessage>

</interface>
```

Listing 6: Interface definition showing all element categories used in real DD interfaces: integer primitives, strings, arrays, composite blocks, and constraint annotations.

```
<interface interfaceversion="3" interfacename="SensorData"
   byte_order="BIG_ENDIAN">
  <InterfaceMessage name="Reading" id="2001" direction="
   NOT_APPLICABLE">
    <UInt32  name="SensorId"    role="required"/>
    <Float32 name="Value"       role="required"/>
    <UInt32  name="Timestamp"   role="required"/>
    <Float32 name="Confidence"  role="optional" default="1.0"
   />
```

```
  </InterfaceMessage>
</interface>
```

Listing 7: The same interface extended with DDCA `role` and `default` attributes. The `Confidence` field is optional and carries a declared default value.

A.2 Interface Object

```
struct FieldDescriptor {
    FieldId     field_id;
    std::string name;
    PrimitiveType type;
    FieldRole   role;
    bool        has_default_value{false};
    std::string default_value;
    size_t      offset{0};
    size_t      size{0};
};

struct MessageDescriptor {
    MessageId   message_id;
    std::string name;
    std::vector<FieldDescriptor> fields;
    size_t      wire_size{0};
};

struct Interface {
    InterfaceId interface_id;
    std::string interface_name;
    std::string interface_version;
    std::string byte_order;
    std::vector<MessageDescriptor> messages;
};
```

Listing 8: C++ structs that make up the interface object produced by the parser.

B Field Operation and Resolution Plan Structs

```
struct FieldOp {
    OpCode  opcode;
    size_t  src_offset{0};
    size_t  dst_offset{0};
    size_t  src_size{0};
    size_t  dst_size{0};
    PrimitiveType src_type{};
    PrimitiveType dst_type{};
    std::vector<std::byte> default_bytes; // pre-baked bytes
   for InjectDefault
};

struct ResolutionPlan {
    PlanId    plan_id;
    DomainId  domain_id;
    MessageId publisher_message_id;
    MessageId subscriber_message_id;
    DomainCompatibilityPolicy policy;
    std::vector<FieldOp> operations;
};
```

Listing 9: Field operation and resolution plan structs as defined in `plan.h`.

C Mixed Interface Pair

```
<?xml version="1.0" encoding="UTF-8"?>
<interface interfacename="SensorNode" interfaceversion="1.0.0
   " byte_order="big_endian">

  <!-- Messages 1001-1005: unchanged between v1.0 and v2.0
   -->

  <InterfaceMessage name="NodeStatus" id="1001">
    <UInt8   name="OperatingMode"        role="required"/>
    <UInt8   name="NodePhase"            role="required"/>
    <UInt16  name="FaultCode"            role="required"/>
    <UInt16  name="SelfTestResult"       role="required"/>
    <UInt16  name="ActiveWarnings"       role="required"/>
    <Float32 name="SupplyVoltageV"       role="required"/>
    <Float32 name="CpuLoadPercent"       role="required"/>
```

```
    <Float32 name="CoreTempC"            role="required"/>
    <UInt32  name="UptimeMs"             role="required"/>
    <UInt8   name="RedundancyStatus"     role="required"/>
    <UInt8   name="SafingStatus"         role="required"/>
    <UInt8   name="MaintenanceFlag"      role="optional"/>
    <UInt16  name="DiagCycleCount"       role="diagnostic"/>
  </InterfaceMessage>

  <InterfaceMessage name="PositionReport" id="1002">
    <Float32 name="Latitude"             role="required"/>
    <Float32 name="Longitude"            role="required"/>
    <Float32 name="AltitudeM"            role="required"/>
    <Float32 name="VelocityNorthMps"     role="required"/>
    <Float32 name="VelocityEastMps"      role="required"/>
    <Float32 name="VelocityDownMps"      role="required"/>
    <Float32 name="GroundSpeedMps"       role="required"/>
    <Float32 name="HeadingDeg"           role="required"/>
    <UInt8   name="PositionSource"       role="required"/>
    <UInt8   name="FixQuality"           role="required"/>
    <UInt8   name="SatelliteCount"       role="required"/>
    <UInt16  name="FaultCode"            role="required"/>
    <UInt8   name="Validity"             role="required"/>
    <UInt16  name="DiagWord"             role="diagnostic"/>
  </InterfaceMessage>

  <InterfaceMessage name="VelocitySensorData" id="1003">
    <Float32 name="IndicatedSpeed"       role="required"/>
    <Float32 name="TrueSpeed"            role="required"/>
    <Float32 name="BaroAltitude"         role="required"/>
    <Float32 name="ClimbRate"            role="required"/>
    <Float32 name="MachNumber"           role="required"/>
    <Float32 name="AngleOfAttack"        role="required"/>
    <Float32 name="SideslipAngle"        role="required"/>
    <Float32 name="AirTempC"             role="required"/>
    <Float32 name="DynamicPressure"      role="required"/>
    <UInt8   name="Validity"             role="required"/>
    <UInt16  name="FaultCode"            role="required"/>
    <UInt8   name="ProbeHeaterStatus"    role="optional"/>
    <UInt16  name="DiagWord"             role="diagnostic"/>
  </InterfaceMessage>

  <InterfaceMessage name="PowerUnitStatus" id="1004">
    <Float32 name="ShaftSpeedLowPct"     role="required"/>
```

```
    <Float32 name="ShaftSpeedHighPct"    role="required"/>
    <Float32 name="ExhaustTempC"         role="required"/>
    <Float32 name="FuelFlowKgH"          role="required"/>
    <Float32 name="OutputForceKn"        role="required"/>
    <Float32 name="LubePressureBar"      role="required"/>
    <Float32 name="LubeTempC"            role="required"/>
    <UInt8   name="OperatingMode"        role="required"/>
    <UInt8   name="ThrottlePosition"     role="required"/>
    <UInt16  name="FaultCode"            role="required"/>
    <UInt8   name="BoostActive"          role="required"/>
    <UInt8   name="TrimStatus"           role="optional"/>
    <UInt16  name="DiagWord"             role="diagnostic"/>
  </InterfaceMessage>

  <InterfaceMessage name="LinkStatus" id="1005">
    <UInt16  name="PrimaryChannelKhz"    role="required"/>
    <UInt16  name="SecondaryChannelKhz"  role="required"/>
    <UInt8   name="RadioMode"            role="required"/>
    <UInt8   name="SquelchLevel"         role="required"/>
    <UInt8   name="TxPowerLevel"         role="required"/>
    <UInt8   name="DataLinkMode"         role="required"/>
    <UInt16  name="NetworkId"            role="required"/>
    <UInt8   name="IdentCode1"           role="required"/>
    <UInt8   name="IdentCode2"           role="required"/>
    <UInt8   name="IdentStatus"          role="required"/>
    <UInt8   name="Validity"             role="required"/>
    <UInt16  name="FaultCode"            role="required"/>
    <UInt8   name="DiagByte"             role="diagnostic"/>
  </InterfaceMessage>

  <!-- Messages 1006-1009: pub v1.0 appends optional fields
   not declared by sub v2.0
       DDCA emits SkipSource for these trailing publisher-
   only fields -->

  <InterfaceMessage name="MotionSensorData" id="1006">
    <Float32 name="AccelXMps2"           role="required"/>
    <Float32 name="AccelYMps2"           role="required"/>
    <Float32 name="AccelZMps2"           role="required"/>
    <Float32 name="GyroRollDegS"         role="required"/>
    <Float32 name="GyroPitchDegS"        role="required"/>
    <Float32 name="GyroYawDegS"          role="required"/>
    <UInt8   name="Validity"             role="required"/>
```

```
    <UInt16  name="FaultCode"            role="required"/>
    <Float32 name="SensorTempC"          role="required"/>
    <UInt8   name="CalibrationStatus"    role="required"/>
    <UInt32  name="RawAdcChannelA"       role="optional"/>
    <UInt32  name="RawAdcChannelB"       role="optional"/>
    <UInt32  name="RawAdcChannelC"       role="optional"/>
    <UInt32  name="RawAdcChannelD"       role="optional"/>
  </InterfaceMessage>

  <InterfaceMessage name="RangeSensorData" id="1007">
    <Float32 name="MeasuredRangeM"       role="required"/>
    <Float32 name="ApproachRateMps"      role="required"/>
    <UInt8   name="Validity"             role="required"/>
    <UInt8   name="ProximityWarningLevel" role="required"/>
    <UInt16  name="FaultCode"            role="required"/>
    <Float32 name="ClearanceM"           role="required"/>
    <UInt8   name="SensorMode"           role="required"/>
    <UInt8   name="CollisionWarning"     role="required"/>
    <UInt16  name="SignalStrengthRaw"    role="optional"/>
    <UInt16  name="NoiseLevelRaw"        role="optional"/>
    <UInt8   name="ReturnQuality"        role="optional"/>
    <UInt8   name="AntennaTiltStatus"    role="optional"/>
  </InterfaceMessage>

  <InterfaceMessage name="ReservoirSystemData" id="1008">
    <Float32 name="TotalVolumeKg"        role="required"/>
    <Float32 name="TankAKg"              role="required"/>
    <Float32 name="TankBKg"              role="required"/>
    <Float32 name="TankCKg"              role="required"/>
    <Float32 name="FlowRateKgH"          role="required"/>
    <UInt8   name="ValveStatus"          role="required"/>
    <UInt8   name="TransferMode"         role="required"/>
    <UInt16  name="FaultCode"            role="required"/>
    <UInt8   name="ValvePosFeedback"     role="optional"/>
    <UInt8   name="PumpStatusRaw"        role="optional"/>
    <UInt16  name="PumpPressureRaw"      role="optional"/>
    <UInt16  name="LevelSensorRaw"       role="optional"/>
  </InterfaceMessage>

  <InterfaceMessage name="PressureSystemData" id="1009">
    <Float32 name="PressureABar"         role="required"/>
    <Float32 name="PressureBBar"         role="required"/>
    <Float32 name="ReservoirLevelA"      role="required"/>
```

```
    <Float32 name="ReservoirLevelB"      role="required"/>
    <UInt8   name="ValveStatus"          role="required"/>
    <UInt8   name="PumpAStatus"          role="required"/>
    <UInt8   name="PumpBStatus"          role="required"/>
    <UInt16  name="FaultCode"            role="required"/>
    <UInt16  name="ActuatorRawA"         role="optional"/>
    <UInt16  name="ActuatorRawB"         role="optional"/>
    <UInt16  name="ActuatorRawC"         role="optional"/>
    <UInt16  name="TempSensorRaw"        role="optional"/>
  </InterfaceMessage>

  <!-- Messages 1010-1013: sub v2.0 adds optional fields not
   sent by pub v1.0
       DDCA emits InjectDefault for those subscriber-only
   fields -->

  <InterfaceMessage name="ActuatorPositions" id="1010">
    <Float32 name="ActuatorAPosDeg"      role="required"/>
    <Float32 name="ActuatorBPosDeg"      role="required"/>
    <Float32 name="ActuatorCPosDeg"      role="required"/>
    <Float32 name="ActuatorDPosDeg"      role="required"/>
    <Float32 name="ActuatorEPosDeg"      role="required"/>
    <Float32 name="ActuatorFPosDeg"      role="required"/>
    <UInt8   name="ControlMode"          role="required"/>
    <UInt8   name="Validity"             role="required"/>
    <UInt16  name="FaultCode"            role="required"/>
  </InterfaceMessage>

  <InterfaceMessage name="ControlLoopState" id="1011">
    <UInt8   name="LoopEngaged"          role="required"/>
    <UInt8   name="ActiveMode"           role="required"/>
    <UInt8   name="ArmedMode"            role="required"/>
    <Float32 name="SetpointAltitude"     role="required"/>
    <Float32 name="SetpointSpeed"        role="required"/>
    <Float32 name="SetpointHeading"      role="required"/>
    <Float32 name="SetpointClimbRate"    role="required"/>
    <UInt16  name="FaultCode"            role="required"/>
    <UInt8   name="Validity"             role="required"/>
  </InterfaceMessage>

  <InterfaceMessage name="PayloadSystemStatus" id="1012">
    <UInt8   name="MasterEnableStatus"   role="required"/>
    <UInt8   name="SelectedPayloadType"  role="required"/>
```

```
    <UInt8   name="StationMask"          role="required"/>
    <UInt8   name="ReleaseAuth"          role="required"/>
    <UInt16  name="UnitsRemaining"       role="required"/>
    <Float32 name="ArmingAltitude"       role="required"/>
    <UInt8   name="SafingStatus"         role="required"/>
    <UInt16  name="FaultCode"            role="required"/>
    <UInt8   name="Validity"             role="required"/>
  </InterfaceMessage>

  <InterfaceMessage name="NetworkPeerStatus" id="1013">
    <UInt8   name="LinkEngaged"          role="required"/>
    <UInt8   name="PeerId"               role="required"/>
    <UInt16  name="LinkQuality"          role="required"/>
    <UInt16  name="TxRatePerSec"         role="required"/>
    <UInt16  name="RxRatePerSec"         role="required"/>
    <UInt8   name="SignalStrengthDbm"    role="required"/>
    <UInt8   name="RoundtripMs"          role="required"/>
    <UInt16  name="FaultCode"            role="required"/>
    <UInt8   name="Validity"             role="required"/>
  </InterfaceMessage>

  <!-- Messages 1014-1017: same fields, different order
   between v1.0 and v2.0
       DDCA emits CopyBytes with adjusted source/destination
   offsets -->

  <InterfaceMessage name="NodeLocationReport" id="1014">
    <Float32 name="Latitude"             role="required"/>
    <Float32 name="Longitude"            role="required"/>
    <Float32 name="Altitude"             role="required"/>
    <Float32 name="Heading"              role="required"/>
    <Float32 name="NearestPeerRange"     role="required"/>
    <Float32 name="NearestPeerBearing"   role="required"/>
    <UInt8   name="PeerCategory"         role="required"/>
    <UInt8   name="NodeMode"             role="required"/>
    <UInt16  name="FaultCode"            role="required"/>
    <UInt8   name="Validity"             role="required"/>
  </InterfaceMessage>

  <InterfaceMessage name="AlertState" id="1015">
    <UInt8   name="PriorityAlertId"      role="required"/>
    <UInt8   name="CriticalAlertFlag"    role="required"/>
    <Float32 name="AlertSourceBearing"   role="required"/>
```

```
    <Float32 name="AlertSourceElevation" role="required"/>
    <UInt8   name="CounterMode"          role="required"/>
    <UInt8   name="CountermeasureArmed"  role="required"/>
    <UInt16  name="SignalCount"          role="required"/>
    <UInt8   name="Validity"             role="required"/>
    <UInt16  name="FaultCode"            role="required"/>
    <UInt8   name="CountermeasureStock"  role="required"/>
  </InterfaceMessage>

  <InterfaceMessage name="ScheduledTaskData" id="1016">
    <UInt8   name="TaskPhase"            role="required"/>
    <UInt8   name="ActiveCheckpointIndex" role="required"/>
    <Float32 name="CheckpointLatitude"   role="required"/>
    <Float32 name="CheckpointLongitude"  role="required"/>
    <Float32 name="CheckpointAltitude"   role="required"/>
    <Float32 name="DistanceToCheckpointM" role="required"/>
    <Float32 name="TimeToCheckpointSec"  role="required"/>
    <UInt16  name="TaskId"               role="required"/>
    <UInt8   name="Validity"             role="required"/>
    <UInt8   name="CheckpointType"       role="required"/>
  </InterfaceMessage>

  <InterfaceMessage name="DeviceConfiguration" id="1017">
    <UInt8   name="PrimaryScanMode"      role="required"/>
    <UInt8   name="ScanPattern"          role="required"/>
    <UInt8   name="PassiveSensorMode"    role="required"/>
    <Float32 name="ScanRangeM"           role="required"/>
    <Float32 name="ScanCentreAngle"      role="required"/>
    <UInt8   name="ScanLayerCount"       role="required"/>
    <UInt8   name="DesignateMode"        role="required"/>
    <UInt16  name="FaultCode"            role="required"/>
    <UInt8   name="Validity"             role="required"/>
    <UInt8   name="ActiveEmitterArmed"   role="required"/>
  </InterfaceMessage>

  <!-- Messages 1018-1019: first six fields are narrow in v1
   .0, widened in v2.0
       DDCA emits ConvertPrimitive for those fields and
   CopyBytes for the rest -->

  <InterfaceMessage name="EnvironmentalSensors" id="1018">
    <UInt8   name="HumidityRaw"          role="required"/>
    <Int16   name="TemperatureTenths"    role="required"/>
```

```
    <UInt8   name="PressureCode"          role="required"/>
    <Int16   name="WindSpeedTenths"       role="required"/>
    <UInt8   name="VisibilityCode"        role="required"/>
    <Int16   name="BaroTrendPa"           role="required"/>
    <Float32 name="AmbientPressureHpa"    role="required"/>
    <Float32 name="ReferencePressureHpa" role="required"/>
    <UInt8   name="IcingFlag"             role="required"/>
    <UInt8   name="PrecipitationType"     role="required"/>
    <UInt16  name="FaultCode"             role="required"/>
    <UInt8   name="Validity"              role="required"/>
  </InterfaceMessage>

  <InterfaceMessage name="NodePerformanceCounters" id="1019">
    <UInt8   name="BurstCount"            role="required"/>
    <Int16   name="ClockOffsetMs"         role="required"/>
    <UInt8   name="RetransmitCount"       role="required"/>
    <Int16   name="JitterUs"              role="required"/>
    <UInt8   name="PeakQueueDepth"        role="required"/>
    <Int16   name="TxLatencyUs"           role="required"/>
    <Float32 name="AvgRoundtripMs"        role="required"/>
    <Float32 name="PeakRoundtripMs"       role="required"/>
    <UInt32  name="TotalTxCount"          role="required"/>
    <UInt32  name="TotalRxCount"          role="required"/>
    <UInt16  name="FaultCode"             role="required"/>
    <UInt8   name="Validity"              role="required"/>
  </InterfaceMessage>

  <!-- Message 1020: all four evolution types in one message
       SkipSource(5): LegacyStateWord1-5 not declared by sub
       CopyBytes(5):  TrackId1-5 reordered in sub
       ConvertPrimitive(5): TempSensorA-E widened to Int16 in
    sub
       InjectDefault(5): HealthBitA-E sub-only with default=0
    -->

  <InterfaceMessage name="NodeSystemSnapshot" id="1020">
    <UInt32  name="LegacyStateWord1"      role="optional"/>
    <UInt32  name="LegacyStateWord2"      role="optional"/>
    <UInt32  name="LegacyStateWord3"      role="optional"/>
    <UInt32  name="LegacyStateWord4"      role="optional"/>
    <UInt32  name="LegacyStateWord5"      role="optional"/>
    <UInt32  name="TrackId1"              role="required"/>
    <UInt32  name="TrackId2"              role="required"/>
```

```
    <UInt32  name="TrackId3"              role="required"/>
    <UInt32  name="TrackId4"              role="required"/>
    <UInt32  name="TrackId5"              role="required"/>
    <Int8    name="TempSensorA"           role="required"/>
    <Int8    name="TempSensorB"           role="required"/>
    <Int8    name="TempSensorC"           role="required"/>
    <Int8    name="TempSensorD"           role="required"/>
    <Int8    name="TempSensorE"           role="required"/>
  </InterfaceMessage>

</interface>
```

Listing 10: Publisher interface `realistic_pub_v1.xml` (SensorNode v1.0.0).

```
<?xml version="1.0" encoding="UTF-8"?>
<interface interfacename="SensorNode" interfaceversion="2.0.0
   " byte_order="big_endian">

  <!-- Messages 1001-1005: unchanged from v1.0 -->

  <InterfaceMessage name="NodeStatus" id="1001">
    <UInt8   name="OperatingMode"        role="required"/>
    <UInt8   name="NodePhase"            role="required"/>
    <UInt16  name="FaultCode"            role="required"/>
    <UInt16  name="SelfTestResult"       role="required"/>
    <UInt16  name="ActiveWarnings"       role="required"/>
    <Float32 name="SupplyVoltageV"       role="required"/>
    <Float32 name="CpuLoadPercent"       role="required"/>
    <Float32 name="CoreTempC"            role="required"/>
    <UInt32  name="UptimeMs"             role="required"/>
    <UInt8   name="RedundancyStatus"     role="required"/>
    <UInt8   name="SafingStatus"         role="required"/>
    <UInt8   name="MaintenanceFlag"      role="optional"/>
    <UInt16  name="DiagCycleCount"       role="diagnostic"/>
  </InterfaceMessage>

  <InterfaceMessage name="PositionReport" id="1002">
    <Float32 name="Latitude"             role="required"/>
    <Float32 name="Longitude"            role="required"/>
    <Float32 name="AltitudeM"            role="required"/>
    <Float32 name="VelocityNorthMps"     role="required"/>
    <Float32 name="VelocityEastMps"      role="required"/>
    <Float32 name="VelocityDownMps"      role="required"/>
```

```
    <Float32 name="GroundSpeedMps"       role="required"/>
    <Float32 name="HeadingDeg"           role="required"/>
    <UInt8   name="PositionSource"       role="required"/>
    <UInt8   name="FixQuality"           role="required"/>
    <UInt8   name="SatelliteCount"       role="required"/>
    <UInt16  name="FaultCode"            role="required"/>
    <UInt8   name="Validity"             role="required"/>
    <UInt16  name="DiagWord"             role="diagnostic"/>
  </InterfaceMessage>

  <InterfaceMessage name="VelocitySensorData" id="1003">
    <Float32 name="IndicatedSpeed"       role="required"/>
    <Float32 name="TrueSpeed"            role="required"/>
    <Float32 name="BaroAltitude"         role="required"/>
    <Float32 name="ClimbRate"            role="required"/>
    <Float32 name="MachNumber"           role="required"/>
    <Float32 name="AngleOfAttack"        role="required"/>
    <Float32 name="SideslipAngle"        role="required"/>
    <Float32 name="AirTempC"             role="required"/>
    <Float32 name="DynamicPressure"      role="required"/>
    <UInt8   name="Validity"             role="required"/>
    <UInt16  name="FaultCode"            role="required"/>
    <UInt8   name="ProbeHeaterStatus"    role="optional"/>
    <UInt16  name="DiagWord"             role="diagnostic"/>
  </InterfaceMessage>

  <InterfaceMessage name="PowerUnitStatus" id="1004">
    <Float32 name="ShaftSpeedLowPct"     role="required"/>
    <Float32 name="ShaftSpeedHighPct"    role="required"/>
    <Float32 name="ExhaustTempC"         role="required"/>
    <Float32 name="FuelFlowKgH"          role="required"/>
    <Float32 name="OutputForceKn"        role="required"/>
    <Float32 name="LubePressureBar"      role="required"/>
    <Float32 name="LubeTempC"            role="required"/>
    <UInt8   name="OperatingMode"        role="required"/>
    <UInt8   name="ThrottlePosition"     role="required"/>
    <UInt16  name="FaultCode"            role="required"/>
    <UInt8   name="BoostActive"          role="required"/>
    <UInt8   name="TrimStatus"           role="optional"/>
    <UInt16  name="DiagWord"             role="diagnostic"/>
  </InterfaceMessage>

  <InterfaceMessage name="LinkStatus" id="1005">
```

```
    <UInt16  name="PrimaryChannelKhz"    role="required"/>
    <UInt16  name="SecondaryChannelKhz"  role="required"/>
    <UInt8   name="RadioMode"            role="required"/>
    <UInt8   name="SquelchLevel"         role="required"/>
    <UInt8   name="TxPowerLevel"         role="required"/>
    <UInt8   name="DataLinkMode"         role="required"/>
    <UInt16  name="NetworkId"            role="required"/>
    <UInt8   name="IdentCode1"           role="required"/>
    <UInt8   name="IdentCode2"           role="required"/>
    <UInt8   name="IdentStatus"          role="required"/>
    <UInt8   name="Validity"             role="required"/>
    <UInt16  name="FaultCode"            role="required"/>
    <UInt8   name="DiagByte"             role="diagnostic"/>
  </InterfaceMessage>

  <!-- Messages 1006-1009: sub v2.0 drops the legacy trailing
    fields from pub v1.0
       DDCA emits SkipSource to advance past those publisher-
   only bytes -->

  <InterfaceMessage name="MotionSensorData" id="1006">
    <Float32 name="AccelXMps2"           role="required"/>
    <Float32 name="AccelYMps2"           role="required"/>
    <Float32 name="AccelZMps2"           role="required"/>
    <Float32 name="GyroRollDegS"         role="required"/>
    <Float32 name="GyroPitchDegS"        role="required"/>
    <Float32 name="GyroYawDegS"          role="required"/>
    <UInt8   name="Validity"             role="required"/>
    <UInt16  name="FaultCode"            role="required"/>
    <Float32 name="SensorTempC"          role="required"/>
    <UInt8   name="CalibrationStatus"    role="required"/>
  </InterfaceMessage>

  <InterfaceMessage name="RangeSensorData" id="1007">
    <Float32 name="MeasuredRangeM"       role="required"/>
    <Float32 name="ApproachRateMps"      role="required"/>
    <UInt8   name="Validity"             role="required"/>
    <UInt8   name="ProximityWarningLevel" role="required"/>
    <UInt16  name="FaultCode"            role="required"/>
    <Float32 name="ClearanceM"           role="required"/>
    <UInt8   name="SensorMode"           role="required"/>
    <UInt8   name="CollisionWarning"     role="required"/>
  </InterfaceMessage>
```

```
<InterfaceMessage name="ReservoirSystemData" id="1008">
  <Float32 name="TotalVolumeKg"        role="required"/>
  <Float32 name="TankAKg"              role="required"/>
  <Float32 name="TankBKg"              role="required"/>
  <Float32 name="TankCKg"              role="required"/>
  <Float32 name="FlowRateKgH"          role="required"/>
  <UInt8   name="ValveStatus"          role="required"/>
  <UInt8   name="TransferMode"         role="required"/>
  <UInt16  name="FaultCode"            role="required"/>
</InterfaceMessage>

<InterfaceMessage name="PressureSystemData" id="1009">
  <Float32 name="PressureABar"         role="required"/>
  <Float32 name="PressureBBar"         role="required"/>
  <Float32 name="ReservoirLevelA"      role="required"/>
  <Float32 name="ReservoirLevelB"      role="required"/>
  <UInt8   name="ValveStatus"          role="required"/>
  <UInt8   name="PumpAStatus"          role="required"/>
  <UInt8   name="PumpBStatus"          role="required"/>
  <UInt16  name="FaultCode"            role="required"/>
</InterfaceMessage>

<!-- Messages 1010-1013: sub v2.0 extends with optional
 fields (default=0)
     DDCA emits InjectDefault for those subscriber-only
 fields -->

<InterfaceMessage name="ActuatorPositions" id="1010">
  <Float32 name="ActuatorAPosDeg"      role="required"/>
  <Float32 name="ActuatorBPosDeg"      role="required"/>
  <Float32 name="ActuatorCPosDeg"      role="required"/>
  <Float32 name="ActuatorDPosDeg"      role="required"/>
  <Float32 name="ActuatorEPosDeg"      role="required"/>
  <Float32 name="ActuatorFPosDeg"      role="required"/>
  <UInt8   name="ControlMode"          role="required"/>
  <UInt8   name="Validity"             role="required"/>
  <UInt16  name="FaultCode"            role="required"/>
  <UInt8   name="AutoCompEnable"       role="optional"
 default="0"/>
  <UInt8   name="CrossCouplingFlag"    role="optional"
 default="0"/>
  <UInt16  name="HealthMonitorWord"    role="optional"
```

```
 default="0"/>
  <UInt16  name="SelfTestWord"          role="optional"
 default="0"/>
</InterfaceMessage>

<InterfaceMessage name="ControlLoopState" id="1011">
  <UInt8   name="LoopEngaged"           role="required"/>
  <UInt8   name="ActiveMode"            role="required"/>
  <UInt8   name="ArmedMode"             role="required"/>
  <Float32 name="SetpointAltitude"      role="required"/>
  <Float32 name="SetpointSpeed"         role="required"/>
  <Float32 name="SetpointHeading"       role="required"/>
  <Float32 name="SetpointClimbRate"     role="required"/>
  <UInt16  name="FaultCode"             role="required"/>
  <UInt8   name="Validity"              role="required"/>
  <UInt8   name="AutoThrottleEnable"    role="optional"
 default="0"/>
  <UInt8   name="EnvelopeLimitEnable"   role="optional"
 default="0"/>
  <UInt8   name="PerfOptMode"           role="optional"
 default="0"/>
  <UInt16  name="ExtDiagWord"           role="optional"
 default="0"/>
</InterfaceMessage>

<InterfaceMessage name="PayloadSystemStatus" id="1012">
  <UInt8   name="MasterEnableStatus"    role="required"/>
  <UInt8   name="SelectedPayloadType"   role="required"/>
  <UInt8   name="StationMask"           role="required"/>
  <UInt8   name="ReleaseAuth"           role="required"/>
  <UInt16  name="UnitsRemaining"        role="required"/>
  <Float32 name="ArmingAltitude"        role="required"/>
  <UInt8   name="SafingStatus"          role="required"/>
  <UInt16  name="FaultCode"             role="required"/>
  <UInt8   name="Validity"              role="required"/>
  <UInt8   name="ReleaseModeOverride"   role="optional"
 default="0"/>
  <UInt8   name="HandoffProtocol"       role="optional"
 default="0"/>
  <UInt16  name="BitStatusWord"         role="optional"
 default="0"/>
  <UInt16  name="LoadoutId"             role="optional"
 default="0"/>
```

```
</InterfaceMessage>

<InterfaceMessage name="NetworkPeerStatus" id="1013">
  <UInt8   name="LinkEngaged"          role="required"/>
  <UInt8   name="PeerId"               role="required"/>
  <UInt16  name="LinkQuality"          role="required"/>
  <UInt16  name="TxRatePerSec"         role="required"/>
  <UInt16  name="RxRatePerSec"         role="required"/>
  <UInt8   name="SignalStrengthDbm"    role="required"/>
  <UInt8   name="RoundtripMs"          role="required"/>
  <UInt16  name="FaultCode"            role="required"/>
  <UInt8   name="Validity"             role="required"/>
  <UInt8   name="EncryptMode"          role="optional"
 default="0"/>
  <UInt8   name="SyncState"            role="optional"
 default="0"/>
  <UInt8   name="PeerCount"            role="optional"
 default="0"/>
  <UInt16  name="BitErrorWord"         role="optional"
 default="0"/>
</InterfaceMessage>

<!-- Messages 1014-1017: same fields as v1.0, reordered for
  struct alignment
     DDCA emits CopyBytes with adjusted source/destination
 offsets -->

<InterfaceMessage name="NodeLocationReport" id="1014">
  <Float32 name="NearestPeerRange"     role="required"/>
  <Float32 name="NearestPeerBearing"   role="required"/>
  <Float32 name="Latitude"             role="required"/>
  <Float32 name="Longitude"            role="required"/>
  <Float32 name="Altitude"             role="required"/>
  <Float32 name="Heading"              role="required"/>
  <UInt8   name="PeerCategory"         role="required"/>
  <UInt8   name="NodeMode"             role="required"/>
  <UInt16  name="FaultCode"            role="required"/>
  <UInt8   name="Validity"             role="required"/>
</InterfaceMessage>

<InterfaceMessage name="AlertState" id="1015">
  <UInt8   name="CriticalAlertFlag"    role="required"/>
  <UInt8   name="CountermeasureArmed"  role="required"/>
```

```
    <UInt8   name="CountermeasureStock"  role="required"/>
    <UInt8   name="CounterMode"          role="required"/>
    <UInt8   name="PriorityAlertId"      role="required"/>
    <Float32 name="AlertSourceBearing"   role="required"/>
    <Float32 name="AlertSourceElevation" role="required"/>
    <UInt16  name="SignalCount"          role="required"/>
    <UInt8   name="Validity"             role="required"/>
    <UInt16  name="FaultCode"            role="required"/>
  </InterfaceMessage>

  <InterfaceMessage name="ScheduledTaskData" id="1016">
    <Float32 name="CheckpointLatitude"   role="required"/>
    <Float32 name="CheckpointLongitude"  role="required"/>
    <Float32 name="CheckpointAltitude"   role="required"/>
    <UInt8   name="TaskPhase"            role="required"/>
    <UInt8   name="ActiveCheckpointIndex" role="required"/>
    <UInt16  name="TaskId"               role="required"/>
    <Float32 name="DistanceToCheckpointM" role="required"/>
    <Float32 name="TimeToCheckpointSec"  role="required"/>
    <UInt8   name="CheckpointType"       role="required"/>
    <UInt8   name="Validity"             role="required"/>
  </InterfaceMessage>

  <InterfaceMessage name="DeviceConfiguration" id="1017">
    <UInt8   name="DesignateMode"        role="required"/>
    <UInt8   name="ActiveEmitterArmed"   role="required"/>
    <UInt8   name="PrimaryScanMode"      role="required"/>
    <UInt8   name="ScanPattern"          role="required"/>
    <UInt8   name="PassiveSensorMode"    role="required"/>
    <Float32 name="ScanRangeM"           role="required"/>
    <Float32 name="ScanCentreAngle"      role="required"/>
    <UInt8   name="ScanLayerCount"       role="required"/>
    <UInt16  name="FaultCode"            role="required"/>
    <UInt8   name="Validity"             role="required"/>
  </InterfaceMessage>

  <!-- Messages 1018-1019: first six fields widened in v2.0
       DDCA emits ConvertPrimitive for widened fields,
   CopyBytes for the rest -->

  <InterfaceMessage name="EnvironmentalSensors" id="1018">
    <UInt16  name="HumidityRaw"          role="required"/>
    <Int32   name="TemperatureTenths"    role="required"/>
```

```
    <UInt16  name="PressureCode"          role="required"/>
    <Int32   name="WindSpeedTenths"       role="required"/>
    <UInt16  name="VisibilityCode"        role="required"/>
    <Int32   name="BaroTrendPa"           role="required"/>
    <Float32 name="AmbientPressureHpa"    role="required"/>
    <Float32 name="ReferencePressureHpa" role="required"/>
    <UInt8   name="IcingFlag"             role="required"/>
    <UInt8   name="PrecipitationType"     role="required"/>
    <UInt16  name="FaultCode"             role="required"/>
    <UInt8   name="Validity"              role="required"/>
  </InterfaceMessage>

  <InterfaceMessage name="NodePerformanceCounters" id="1019">
    <UInt16  name="BurstCount"            role="required"/>
    <Int32   name="ClockOffsetMs"         role="required"/>
    <UInt16  name="RetransmitCount"       role="required"/>
    <Int32   name="JitterUs"              role="required"/>
    <UInt16  name="PeakQueueDepth"        role="required"/>
    <Int32   name="TxLatencyUs"           role="required"/>
    <Float32 name="AvgRoundtripMs"        role="required"/>
    <Float32 name="PeakRoundtripMs"       role="required"/>
    <UInt32  name="TotalTxCount"          role="required"/>
    <UInt32  name="TotalRxCount"          role="required"/>
    <UInt16  name="FaultCode"             role="required"/>
    <UInt8   name="Validity"              role="required"/>
  </InterfaceMessage>

  <!-- Message 1020: all four evolution types in one message
       SkipSource(5): LegacyStateWord1-5 not declared here
       CopyBytes(5):  TrackId1-5 reordered [3,1,4,2,5]
       ConvertPrimitive(5): TempSensorA-E widened from Int8
       InjectDefault(5): HealthBitA-E sub-only with default=0
    -->

  <InterfaceMessage name="NodeSystemSnapshot" id="1020">
    <UInt32  name="TrackId3"              role="required"/>
    <UInt32  name="TrackId1"              role="required"/>
    <UInt32  name="TrackId4"              role="required"/>
    <UInt32  name="TrackId2"              role="required"/>
    <UInt32  name="TrackId5"              role="required"/>
    <Int16   name="TempSensorA"           role="required"/>
    <Int16   name="TempSensorB"           role="required"/>
    <Int16   name="TempSensorC"           role="required"/>
```

```
    <Int16   name="TempSensorD"          role="required"/>
    <Int16   name="TempSensorE"          role="required"/>
    <UInt8   name="HealthBitA"           role="optional"
   default="0"/>
    <UInt8   name="HealthBitB"           role="optional"
   default="0"/>
    <UInt8   name="HealthBitC"           role="optional"
   default="0"/>
    <UInt8   name="HealthBitD"           role="optional"
   default="0"/>
    <UInt8   name="HealthBitE"           role="optional"
   default="0"/>
  </InterfaceMessage>

</interface>
```

Listing 11: Subscriber interface `realistic_sub_v2.xml` (SensorNode v2.0.0).

D Full Correctness Check List

Table 4 lists the 30 compatibility checks from Part 1 of the correctness benchmark (15 evolution scenarios evaluated under the Exact and Flexible policies). Table 5 lists the 39 value correctness checks from Part 2 (four operation types and three combination scenarios). All 69 checks pass.

Evolution scenario	Policy	Expected	Result
Identical schema	Exact	Accept	Pass
Identical schema	Flexible	Accept	Pass
Publisher: extra required field	Exact	Reject	Pass
Publisher: extra required field	Flexible	Accept	Pass
Publisher: extra optional field	Exact	Reject	Pass
Publisher: extra optional field	Flexible	Accept	Pass
Publisher: extra diagnostic field	Exact	Reject	Pass
Publisher: extra diagnostic field	Flexible	Accept	Pass
Subscriber: required field missing	Exact	Reject	Pass
Subscriber: required field missing	Flexible	Reject	Pass
Subscriber: optional field with default	Exact	Reject	Pass
Subscriber: optional field with default	Flexible	Accept	Pass
Subscriber: diagnostic field	Exact	Reject	Pass
Subscriber: diagnostic field	Flexible	Accept	Pass
Field reordering	Exact	Reject	Pass
Field reordering	Flexible	Accept	Pass
Safe primitive widening	Exact	Reject	Pass
Safe primitive widening	Flexible	Accept	Pass
Unsafe type narrowing	Exact	Reject	Pass
Unsafe type narrowing	Flexible	Reject	Pass
Incompatible type change	Exact	Reject	Pass
Incompatible type change	Flexible	Reject	Pass
Combo: reordering and publisher extra	Exact	Reject	Pass
Combo: reordering and publisher extra	Flexible	Accept	Pass
Combo: reordering and subscriber optional	Exact	Reject	Pass
Combo: reordering and subscriber optional	Flexible	Accept	Pass
Combo: widening and subscriber optional	Exact	Reject	Pass
Combo: widening and subscriber optional	Flexible	Accept	Pass
Optional field with invalid default	Exact	Reject	Pass
Optional field with invalid default	Flexible	Accept	Pass

Table 4 Compatibility decision checks (Part 1).

Scenario	Assertion	Result
CopyBytes	Registration accepted	Pass
CopyBytes	Fields found in interface object	Pass
CopyBytes	Field1 (UInt32) value preserved (0xDEAD)	Pass
CopyBytes	Field2 (Float32) value preserved (1.5)	Pass
SkipSource (pub optional)	Registration accepted	Pass
SkipSource (pub optional)	Fields found in interface object	Pass
SkipSource (pub optional)	Adapted size equals subscriber wire size	Pass
SkipSource (pub optional)	Base field value preserved after adaptation	Pass
SkipSource (pub optional)	Extra publisher bytes absent from adapted output	Pass
SkipSource (pub required)	Registration accepted	Pass
SkipSource (pub required)	Fields found in interface object	Pass
SkipSource (pub required)	Adapted size equals subscriber wire size	Pass
SkipSource (pub required)	Field1 value preserved after adaptation	Pass
SkipSource (pub diagnostic)	Registration accepted	Pass
SkipSource (pub diagnostic)	Fields found in interface object	Pass
SkipSource (pub diagnostic)	Adapted size equals subscriber wire size	Pass
SkipSource (pub diagnostic)	Field1 value preserved after adaptation	Pass
InjectDefault	Registration accepted	Pass
InjectDefault	Publisher field found in interface object	Pass
InjectDefault	Base field value preserved after adaptation	Pass
InjectDefault	Injected field equals declared default (0)	Pass
ConvertPrimitive	Registration accepted	Pass
ConvertPrimitive	Fields found in interface object	Pass
ConvertPrimitive	UInt8(99) widened to UInt16(99)	Pass
ConvertPrimitive	Non-widened field value preserved	Pass
Combo: CopyBytes + SkipSource	Registration accepted	Pass
Combo: CopyBytes + SkipSource	Fields found in interface object	Pass
Combo: CopyBytes + SkipSource	FieldA value at correct subscriber offset	Pass
Combo: CopyBytes + SkipSource	FieldB value at correct subscriber offset	Pass
Combo: CopyBytes + InjectDefault	Registration accepted	Pass
Combo: CopyBytes + InjectDefault	Fields found in interface object	Pass
Combo: CopyBytes + InjectDefault	FieldA value preserved at new offset	Pass
Combo: CopyBytes + InjectDefault	FieldB value preserved at new offset	Pass
Combo: CopyBytes + InjectDefault	OptC equals injected default (42)	Pass
Combo: ConvertPrimitive + InjectDefault	Registration accepted	Pass
Combo: ConvertPrimitive + InjectDefault	Fields found in interface object	Pass
Combo: ConvertPrimitive + InjectDefault	FieldA UInt8(127) widened to UInt16	Pass
Combo: ConvertPrimitive + InjectDefault	FieldB UInt16(0x1234) widened to UInt32	Pass
Combo: ConvertPrimitive + InjectDefault	OptC equals injected default (7)	Pass

Table 5 Value correctness checks (Part 2). Each check verifies a specific field in the adapted subscriber payload against a known sentinel value.